

START FROM NOWHERE

The Build Playbook

How one person and a set of AI sessions built a multi-exam test-prep platform, every defect hit on the way, and the recipe for doing it again for something else

Built 2026-09-22 from commit 667cf8f · 75 commits over 36 days · 83 recorded defects · 17 chapters

This document is generated. Edit the chapters in docs/playbook/ and the ledger in data/playbook/incidents.jsonl, then run `python3 src/build_playbook.py`.

1. How to Use This Book
2. The Stack, Priced, and What It Would Cost in Open Source
3. From an Empty Repository to a Live Site
4. The Build System
5. The Interface: One System, Defined Once
6. The Algorithm: Adaptive Selection and Honest Scoring
7. Content at Scale, Without Lying About It
8. The Data Layer
9. Payments
10. Business Intelligence
11. Tests and Guards
12. Running the Build as an AI Loop
13. What the Ledger Says About Itself
14. The Defect Ledger
15. The Checklist
16. Adapting This to a Different Business
17. The Bootstrap Pack: Starting a New Project With This Loaded

How to Use This Book

This is the record of building one platform, written so it can be used to build a different one.

The platform is Start From Nowhere, a test-preparation site with five adaptive exam trainers, a college and business-school rankings library, a blog, a forum, subscriptions through two payment processors, and an admin console. It was built between 2026-08-17 and 2026-09-22, which is 36 days, across 75 commits, by one owner directing a series of AI coding sessions. As of this build it is 53 Python files, 101 JavaScript files, 24 TypeScript edge functions, 35 migrations and 63 documents: 1977 tracked files in total.

None of those numbers were typed. They are measured from the repository every time this document is built, which is the first thing worth copying.

What is in here

The book has four kinds of chapter, and they age differently.

The recipe chapters say how to stand up each part of the system: the infrastructure, the build, the interface, the algorithm, the content pipeline, the database, payments, business intelligence. These are the slowest to rot, because they are mostly about sequence and tradeoff rather than about any particular API.

The defect ledger is every bug, error and misfire that cost real time, with what was seen, why it happened, how it surfaced, what fixed it, and what now stops it recurring. Each entry ends with a lesson stated without reference to this codebase, because that is the part that transfers.

The analysis chapter is computed from the ledger. It counts how defects were actually found, which failure modes dominate, and which guards have already failed more than once. A handwritten guide cannot contain this chapter, because it has to be recounted every time the ledger changes.

The bootstrap pack is the part you use to start something new. It is a set of ready-to-paste artefacts for opening a fresh Claude project on a different business idea, with everything this build learned already loaded.

How to actually use it

Starting something new. Go straight to the last chapter, The Bootstrap Pack. It gives you three files and the order to use them in. You do not need to read the rest first; the pack carries the distilled rules.

Building a specific part. Read the matching recipe chapter, then read the ledger entries for that area. The recipe says what to do; the ledger says what will go wrong. The second is worth more.

Reviewing work before it ships. Use The Checklist. It is generated from the ledger, so every line on it exists because something went wrong once, and nothing is on it for completeness.

How this document stays true

It is built, not maintained. Three sources go in:

1. `docs/playbook/*.md` , the prose. Written by a person; this is the judgement.
2. `data/playbook/incidents.jsonl` , the defect ledger. One JSON record per incident,

carrying the commit that fixed it and the file that now guards it.

1. The repository itself, harvested at build time for every figure in the text.

`python3 src/build_playbook.py` assembles them and writes Markdown, HTML, PDF and Word.

Three things fail the build rather than shipping quietly:

- An unresolved `{{PLACEHOLDER}}` , which means a fact the document wanted no longer exists.

- An incident citing a commit hash or a guard file that cannot be found.
- An em dash or en dash, because the house style bans them and the document about the

house rules should not break them.

That last one is a joke with a serious point behind it. A rule that is only enforced by attention is a rule that decays. Every rule worth having should have something that notices when it is broken.

Adding to it

When something breaks, add a record to `data/playbook/incidents.jsonl` before you fix it, while you still remember what you believed was true five minutes ago. That belief is the most valuable field and the first one you lose.

The record needs: what was seen, why, how it surfaced, the fix, the guard that now catches it, and the lesson stated so it makes sense to someone who has never seen this code. The `guard` field is what makes the ledger adaptive: the checklist chapter is generated from those fields, so writing the record is the only step. Nothing has to be copied anywhere.

If two records name the same guard, the analysis chapter will say so, and that means the guard did not hold and needs rebuilding rather than trusting.

The Stack, Priced, and What It Would Cost in Open Source

The whole platform runs for roughly the price of a couple of lunches a month. That is not a boast, it is the design constraint that produced most of the interesting decisions in this book, and it is reproducible.

What is actually running

LAYER	CHOICE	WHY THIS ONE	MONTHLY AT THIS SCALE
Hosting and CDN	Cloudflare Workers with static assets	Global edge, generous free tier, and a real runtime in front of the files when you need one	\$0 to \$5
Domain	Registrar of choice at cost	Nothing clever here	about \$1 amortised
Database, auth, storage	Supabase (managed Postgres)	Row Level Security means the database enforces authorisation, not the app	\$0 to \$25
Serverless functions	Supabase Edge Functions (Deno)	Same project, same secrets store, no second vendor	included
Payments	Stripe Checkout, Apple In-App Purchase	Checkout removes PCI scope entirely; Apple is mandatory if you ship an iOS app	percentage of revenue
Email	Transactional provider of choice	Deliberately last. Do not build this before you have users	\$0 at low volume
Source and CI	GitHub with Actions	The deploy hook and the test runner live in the same place as the code	\$0
Error and product telemetry	Written in house, about 200 lines	Covered below; this is the surprising one	\$0
Analytics	First-party beacon, written in house	Covered below	\$0
Charts	Written in house, one file	Covered below	\$0

Total fixed cost at the scale this platform runs: under thirty dollars a month, most of which is the database.

The three things not to buy

These are the decisions that saved the most money and, more importantly, the most complexity. Each replaced a product that would have been reasonable to buy.

Analytics. A first-party beacon is a `fetch` on page load carrying path, referrer, UTM parameters, a session id from `sessionStorage`, device class and time on page, into one table. Country and a salted address hash are added by a database trigger so the raw address is never stored. That is a few dozen lines. What you get that you cannot buy: the data is in your own database next to your users, so a question like "which landing pages precede a subscription" is a join rather than an integration project. What you give up: nothing an ad blocker was not already taking from the paid product.

Error monitoring. A global `error` and `unhandledrejection` handler posting to an insert-only table, with a clustering view and a human triage queue. The clustering matters more than the capture: raw errors are noise, and the value is in a human marking each cluster real, fixed, will not fix, or noise, so the queue gets quieter as it learns rather than louder. Nothing in it edits code, and that is deliberate.

Charts. A charting library is two hundred kilobytes and a second thing to pin in your Content Security Policy. Five chart forms plus a stat tile, built as inline SVG from numbers, is about six hundred lines and smaller than the smallest library's minified core. The chapter on the interface explains why the colours in it are computed rather than chosen.

The rule behind all three: **buy the thing that would be dangerous to get wrong, build the thing that is mostly a shape.** Payments are dangerous. Authentication is dangerous. A bar chart is a shape.

The open-source equivalent of every paid piece

If the managed services are unavailable, too expensive, or politically impossible, here is the substitution, and honestly what it costs you.

MANAGED	OPEN SOURCE	WHAT CHANGES
Cloudflare Workers	Caddy or nginx on a small VPS; Nixpacks or Dokku for deploys	You now own TLS renewal, log rotation and the box. Budget half a day to set up and an hour a month forever
Supabase	Postgres plus PostgREST plus GoTrue, or the self-hosted Supabase compose file	You get the same API surface. You also get backups, upgrades and connection pooling as your job. This is the biggest jump in operational load of anything in this table
Supabase Edge Functions	Deno Deploy, or a plain Node or Deno process behind the reverse proxy	Straightforward. The awkward part is secrets, which you will end up managing with something like SOPS or a mounted env file
Stripe Checkout	There is no real substitute, and you should not want one	Self-hosting payments means PCI scope. Do not
GitHub Actions	Forgejo Actions, Woodpecker, or Drone	Fine. Slower to set up, one fewer vendor
Managed Postgres backups	pgBackRest or WAL-G to object storage	Works well and is genuinely worth learning even on a managed service, because it is also your migration path off it
Supabase Auth	Keycloak, Authentik, Ory Kratos	All three are more capable and considerably more work. Keycloak in particular is a serious piece of infrastructure
Cloudflare Web Analytics	Plausible, Umami, Matomo, all self-hostable	Umami is the lightest. But see the point above: once you have a first-party beacon into your own database, a second analytics product is mostly duplication
Playwright	Already open source	No change
Anything for charts	ECharts, Chart.js, Observable Plot	All good. Weigh the bundle against the six hundred lines
Anything for PDF	Headless Chromium print-to-PDF, WeasyPrint, Typst	Chromium is already on the machine if you run browser tests. That is how this document is

MANAGED	OPEN SOURCE	WHAT CHANGES
		produced

The honest summary of that table: the only irreplaceable paid component is the payment processor. Everything else is a convenience purchase, and the convenience is real but bounded. The thing you are actually buying from a managed database is not Postgres; it is somebody else being on call for it.

What the architecture buys you

Two structural choices shaped everything else, and both are worth copying.

Static pages, built ahead of time, with no runtime framework. Every page on the site is a self-contained HTML file with inline CSS and inline JavaScript, generated by a Python build. There is no server rendering, no hydration and no client-side router. The consequences compound: pages are fast because there is nothing to boot; the whole site can be served from a CDN for nothing; there is no dependency tree to keep patched; and the security surface of a page is what you can read in that page. The cost is that anything dynamic has to be deliberate, which turns out to be a feature.

The database enforces authorisation, not the application. Row Level Security on every table, and every privileged read behind a `SECURITY DEFINER` function that checks an admin table. The app cannot grant itself anything, because the app is a browser, and a browser is not trusted. This is what makes it safe to ship a site with no backend of your own.

Both decisions have the same shape: **push the guarantee down to the layer that cannot be bypassed.** A rule enforced in JavaScript is a suggestion.

From an Empty Repository to a Live Site

The order below is the one that worked, and the order matters more than any individual step. Most of what went wrong in this build went wrong because something was done before the thing that would have caught it.

Phase 0: the rules file, before any code

Write the project's standing rules into a file the AI session reads on every turn. On this project that is `CLAUDE.md`, 63 documents in, and it is still the highest leverage file in the repository.

It is not documentation. It is the constitution, and it should contain only things that are load-bearing and that you would otherwise have to say again:

- The house style rules that a build can enforce. Ours bans em dashes and en dashes

outright, everywhere, including commit messages. That sounds trivial. It means a single grep decides whether copy was written by hand or pasted from somewhere, and the build fails on a violation.

- The sourcing rules. Ours: never state a statistic, price or external fact from memory;

every published figure carries a source, a year and a URL; unverifiable means null, never a guess; and a named list of banned sources.

- **Never invent a value.** Every identifier, hash, id, count, price, date and URL that

goes into a tool call, a commit or a page must be read from real output first. This rule exists on this project because a forty-character hex string was once typed straight into a merge call as though it were a commit SHA. The API rejected it as malformed, which was luck. A well-formed guess would have merged something else.

- The communication rule. Ours: if a link or a service cannot be reached, say so

immediately and ask, before doing the work another way. Never quietly substitute a screenshot or a memory for the source that was pointed to.

Every one of those was written after something went wrong. Start with them anyway.

Phase 1: the build, before the content

Write the thing that turns sources into pages before you have many pages. Ours is `src/build.py`, and on day one it did almost nothing. What matters is that it exists, so that every guard you later need has somewhere to live.

By the end it does all of this, and each item was added the day it was needed:

- Injects shared header, footer and CSS into every page from one place
- Parses every inline script with node and fails on a syntax error
- Fails on an em or en dash in any hand-edited document or item bank
- Fails on any asset larger than the platform's per-file limit
- Counts what it produced and asserts the counts agree across artefacts
- Writes the sitemap from the same directory scan that writes the pages

None of these are clever. All of them exist because of an entry in the defect ledger.

Phase 2: one page, deployed, for real

Deploy the simplest possible page to the real domain before building anything else. Not a staging environment: the actual production host, on the actual domain, through the actual pipeline.

Everything you learn here is cheap now and expensive later. The per-file size limit. Which directories the asset uploader walks. Whether your headers file applies to responses your own code generates.

Whether `www` and the apex are the same origin. That last one cost this project a split consent state and a diluted ranking signal, and it is ten lines of redirect in the right place.

Phase 3: the data layer, with the guarantees tested

Before the interface, get the database right, because it is the thing you cannot casually change later.

1. Every table has Row Level Security enabled, from creation. Not later.
2. A table with no policy denies everyone, which is the correct default for anything only

your server should touch.

1. Every privileged read goes through a `SECURITY DEFINER` function that checks admin

membership. Clients never read a privileged table directly.

1. Revoke from `PUBLIC` as well as from `anon` and `authenticated`. Revoking from the roles

you can name leaves `PUBLIC` holding the grant, and it will not show up in the place you think to look.

1. Test the guarantee, not the code. Write a SQL smoke test that asserts what must be true

and rolls itself back. Two trigger bugs on this project were found exactly that way and would not have been found by reading.

Phase 4: the interface system, before the interfaces

One file that defines the type scale, the tracking ramp, the spacing grid, one page width, one gutter and one reading measure, injected everywhere. On this project that is `src/partials.py`, currently 57 tokens.

The reason to do this early is arithmetic. This site reached thirty-seven font sizes, twenty-eight spacing values and eight page widths before the system landed, and pulling them back to one scale was a multi-day job that touched everything. Doing it on day one costs an hour.

Phase 5: the product

Now build the thing. By this point the build catches syntax errors, the database refuses unauthorised reads, the type system stops the interface drifting, and you can deploy.

Phase 6: instrumentation, before growth

Before spending anything on acquisition, be able to answer: how many people arrive, what they do, where they stop. That is the beacon, the funnel and the error queue. All three are small. All three are useless retroactively, which is the entire argument for doing them before you need them.

Phase 7: payments

Last, and only when there is something worth paying for. The payments chapter has the details. The one scheduling note: the event ledger that lets you answer "how much revenue arrived and how much left" has to exist **before** the first real subscriber, because it cannot be reconstructed afterwards from a table you overwrite on every webhook. This project learned that in the right order by luck rather than judgement.

The shape of the whole thing

Read backwards, the order is: make failure loud, make authorisation impossible to bypass, make the interface consistent, then build features, then measure, then charge. Each phase makes the next one cheaper to get wrong.

The Build System

The build is the most valuable thing in the repository, and it is the part people skip.

What it is

`python3 src/build.py` takes sources in `src/` and `data/` and writes every page of the site. Nothing in the deployed output is hand-edited; the generated directories are gitignored so there is no temptation. Sources of truth live in exactly one place, and the build is the only thing that reads them.

The build currently produces five exam trainer applications, a landing page, a rankings library of 94 business schools and a separate college library, exam guide pages, pricing, a blog, a forum shell, and the legal pages. It also produces this document.

Why a build rather than a framework

A framework gives you composition and gives up legibility. A build gives you a directory of finished files you can open, diff, grep and serve from anything. At this scale that trade is heavily in favour of the build, for a specific reason: **the output is inspectable, so guards can run on the output.**

That turns out to be where most defects were caught. Not in the source, in the artefact. Reading the built CSS rather than the template is what eventually found a nested comment that had killed the palette on over a thousand pages. Reading the built page is what found a stale count in a meta description that Google was showing.

The guards, and what each one is for

Every one of these exists because of an incident. They are listed here in the order they would catch a problem.

Parse every generated script. The build writes JavaScript into HTML. It runs node over each inline script and refuses to write the page on a syntax error. One unescaped quote inside a `onclick` string once took down the entire trainer, because a parse error kills the whole script, not the one handler.

Ban the characters you said you would ban. Em dashes and en dashes fail the build in any hand-edited document or item bank. It is a one-line check that makes a style rule real.

Assert the size of every collection. Item counts per category, page counts, sitemap entry counts, school counts. Not the contents, the size. Deletion by shadowing, a filter dropping rows, a loop over an empty list: none of these raise anything, and all of them show up instantly as a count that moved.

Assert how many things you checked. A guard that matched `\d{2,4}` silently stopped checking the two largest item counts when they passed ten thousand, and kept passing. Any guard that selects a subset should report the size of the subset it selected.

Assert the platform's hard limits. Cloudflare rejects a static asset over 25 MiB. The build now fails on any asset over that, because a deploy-time rejection is an expensive place to learn a number your build already knew.

Compare inputs to outputs at every filter. A length filter measuring the wrong unit silently dropped sixteen valid items. Every stage that can reject should say how many it rejected.

Derive, never type. Any number in copy that describes the size of something is computed from that thing at build time. A typed number has a half-life.

Reproducibility

Three consecutive builds must produce identical output. This is worth enforcing because the failure is subtle: this project's item generator seeded from Python's `hash()`, which is randomised per process, so every build produced a different bank and the pinned statistics in the test file drifted run to run. It looked like a flaky test. It was a broken promise. `zlib.crc32` fixed it.

If your build has any randomness, seed it from something stable and assert the reproducibility, or you will eventually delete a test that was telling you the truth.

The test ladder above the build

The build is the first rung. Above it, in the order they run:

- `src/review_bot.js`
- `src/smoke.js`
- `src/smoke_billing.js`
- `src/smoke_business.js`
- `src/smoke_charts.js`
- `src/smoke_consent.js`
- `src/smoke_fit.js`
- `src/smoke_funnel.js`
- `src/smoke_ios.js`
- `src/smoke_items.js`
- `src/smoke_launch.js`
- `src/smoke_load.js`

- `src/smoke_offline.js`
- `src/smoke_pages.js`
- `src/smoke_playbook.js`
- `src/smoke_rankings.js`
- `src/smoke_redirect.js`
- `src/smoke_sharing.js`
- `src/smoke_signup.js`
- `src/test.js`
- `src/weekly_audit.js`

21 test files in total. The layering is deliberate:

1. **The build** catches structural problems in the artefact.
2. **Engine tests** run the domain logic headlessly, once per exam.
3. **Smoke tests** drive a real browser against the real built site at desktop and phone

width, and assert behaviour rather than markup.

1. **Review bots** play the product adversarially and report what only use reveals.
2. **A weekly audit** re-checks contrast, links and metadata across every page.

Rung three is where the most valuable failures show up, and rung four is where the embarrassing ones do.

The Interface: One System, Defined Once

The type and spacing system

One file holds the whole visual system and is injected into every page. Currently 57 tokens covering:

- A type scale, `--t-100` to `--t-900`
- A tracking ramp, `--tr-900` to `--tr-caps`
- A four-pixel spacing grid, `--s1` to `--s10`
- One page width, one gutter, one reading measure
- The full colour palette, in both themes

Reach for a token before a number. A raw pixel value in a template is how this site reached thirty-seven font sizes, twenty-eight spacing values and eight page widths.

Two type rules worth stealing outright:

Type tightens as it grows and never opens up. Negative tracking on headings, scaling from about -0.032em at hero size to 0 at body size. The single positive value is $+0.06\text{em}$, reserved for small uppercase labels.

Three faces, no more. A serif for headings, one sans for everything else, and a mono for figures that sit in a column, never for words. This site once ran a fourth face as a second sans, and the two were close enough that the result read as a mistake rather than as a choice. Sites that do this well are lopsided on purpose: Stripe runs roughly a hundred declarations of its sans to two of mono.

The layout traps

Three CSS behaviours caused more wasted hours on this project than anything else. They are not obscure. They are just silent.

Grid and flex children need `min-width: 0`. Without it a child holds its min-content width and pushes the page sideways on a phone. This broke pricing, the rankings controls and the blog submit form, separately, each time looking like a different bug.

CSS comments do not nest. The first `*/` closes the outer comment. A template placeholder left inside an opening comment killed the colour palette on over a thousand pages and was chased as a design problem for hours.

An undefined custom property is silent. CSS does not treat `var(--nope)` as an error; it falls back and paints. A page here referenced nine colour tokens that existed nowhere, and the legally required Do

Not Sell button rendered as white text on a transparent background on a white page, invisible from the day it shipped. **Audit computed colour on the rendered page, not authored colour in the source.**

And one more, for completeness: `overflow: hidden` makes an element the scroll container that `position: sticky` resolves against. A sticky header that will not stick almost always means an ancestor picked up an `overflow`.

Dark mode, and the way it actually breaks

A dark mode does not break by having a wrong colour. It breaks by **missing** one. A token defined only inside a media query falls back to whatever it inherits, and the result is usually legible enough to pass a glance and wrong enough to matter.

The structure that avoids it:

1. Define the **complete** palette on bare `:root`. Every token, no exceptions.
2. Redefine **only** those tokens under `@media (prefers-color-scheme: dark)`, guarded as

```
:root:not([data-theme="light"])
```

1. Redefine them again under `:root[data-theme="dark"]`, so an explicit choice wins in

both directions.

Three states, not two: light, dark, and follow the system. A two-state toggle cannot express "follow the system", and that is the most common way a dark mode is got wrong.

Then test it by parity: read both `:root` blocks and assert every colour token defined in one is defined in the other. Select by whether the **value** is a colour, not by whether the **name** starts with a colour-ish prefix. Classifying by name caught a font size on this project.

Finally, resolve the theme before first paint, in a tiny inline script in the head. A theme applied after paint is a flash of the wrong one.

Colour in data display

This is the part most teams get wrong, and it is measurable rather than a matter of taste.

Brand colours are usually wrong inside a chart. This site's navy, teal and secondary navy all sit below the chroma floor and read as grey in a plot; its blue against its violet is a deuteranopia delta-E of 0.4, which means two series a colourblind reader cannot distinguish at all. Brand colour belongs on the page around the chart. Inside the plot, the job is telling series apart.

Validate the palette under simulated colour vision deficiency, across every pair, not just neighbours. The categorical set here passes protanopia, deuteranopia and tritanopia on all pairs in light mode.

A dark palette has about half the lightness range to work with. Roughly 0.48 to 0.67, against 0.43 to 0.77 for light, so hue has to carry more of the separation, and hue is exactly what collapses under deuteranopia. Measured, this palette tops out at three slots distinguishable by colour alone on dark. That is only workable with secondary encoding, so secondary encoding is not optional: every chart with two or more series ships a legend, small sets are direct labelled, and every chart has a table view.

Status colours are reserved and never used for a series. A chart with four series and a red one implies the red one is bad.

Never encode identity by colour alone. Two games on this site signalled right and wrong only by colour until an accessibility pass; outcomes now carry a word as well, which also survives greyscale printing and forced-colors mode.

Chart form is a claim

The form you choose asserts something about the data.

A line claims the values in between existed. Money movement on this site was charted as a line, and the line between \$13.32 on one day and \$4.99 on the next passed through every value in between, none of which happened. Discrete events are bars.

A funnel drawn as a tapering polygon encodes the number twice, once in width and once in area, and area is the one people misread. A ranked list of bars with the drop-off written between them says the same thing without the illusion.

Every chart should be able to become its own numbers. A table view under each chart is the accessibility backstop, and it is also what a person actually wants when they are trying to copy a figure out.

Accessibility, as a set of habits

- Announce outcomes in a live region, not just in colour or motion.
- A flipped card, a revealed tile, a state change: say what it is.
- Contrast is measured on the rendered page, in both themes, on every page. A near miss

is a fail; the grey carrying most of the secondary text here measured 4.83:1 on white and 4.41:1 on the tinted cards, and only the second one is obviously wrong.

- No horizontal scroll at 390 pixels. Assert it in a test rather than checking it by eye, and measure the component, not the document, or a wide child can hide behind a narrower page.

The Algorithm: Adaptive Selection and Honest Scoring

This chapter is about a test-preparation engine, but the transferable part is the last section, and it applies to any product that shows a person a number about themselves.

The shape of the engine

One engine serves unknown exams (see `src/engine.js`). Adding an exam is a registry entry plus a tagged bank, not a fork. That was not free: the single most useful refactor in the project was making the section keys, skill lists and copy resolve from an exam id injected at build time, instead of being hardcoded. Before it, one application had been shipping a score card headed with the wrong exam's name and telling students to calibrate at the wrong test maker's site.

If you have two of something, resolve the differences from data, not from conditionals. A conditional on `if (exam === 'sat')` treats "not SAT" as "the original one", and that assumption breaks silently the moment there is a third.

Ability estimation

Three-parameter logistic item response theory, fitted by Fisher scoring, producing an ability estimate and a standard error. Alongside it, an Elo-style per-skill rating that moves faster and is what the learner actually sees at the skill level.

The two exist for different jobs and it is worth being clear about which is which. The IRT estimate is for the score band, because it has a defensible error term. The Elo rating is for selection and for the skill table, because it responds quickly enough to feel like it is paying attention.

Item selection, and the two bugs in it

Selection targets an item near the learner's current ability, with a damping factor so a run of luck does not send the difficulty somewhere absurd.

Bug one: selecting on a per-skill counter instead of measured ability. Early on, a skill with fewer than six answers targeted a fixed starting rating rather than the learner's measured ability in that section. A strong student answering a new skill got beginner items. The fix targets the section's measured ability until the skill has its own evidence.

Bug two: re-serving items the learner had already answered. The candidate pool did not exclude seen items, so a short session could serve the same question twice. The fix filters to unseen candidates and falls back to the full pool only when unseen is empty. The same fix had to be applied separately to passage groups, because a passage drags its sibling questions along with it and the group was not being filtered.

Both were found by a **review bot**: a script that sits the exam repeatedly against the real engine and reports what only playing reveals. It measures estimate recovery, band coverage across ability levels, whether selection actually adapts, bank reach, starving skills, and whether a deliberately planted weakness is diagnosed. That last one is the sharpest test in the suite, because it has a known right answer.

Measuring a stochastic system on one seed is an anecdote. A change here looked like it dropped band coverage from 88 percent to 72. Across three seeds the baseline was 52, 88 and 80 against 72, 80 and 72 for the change. The spread between seeds was larger than the effect, and the regression did not exist.

Scoring, and the honesty constraints

This is the transferable part.

The product reports an estimated **range** on the exam's own scale, never a single number, never called a predicted score, and never shown before a minimum evidence threshold. The band half-width is the standard error of the ability estimate with a floor, so it can narrow with evidence but never collapses to a point.

Four rules sit behind that, and every one of them is a rule about not overclaiming:

1. **A range, not a number.** A single number implies a precision the estimate does not

have. The interval is the honest object.

1. **A floor on the interval.** Even with a lot of evidence, the interval never gets

narrower than the floor, because the model's error is not the only error.

1. **Nothing before the evidence threshold.** An estimate from four answers is noise with

a decimal point.

1. **The method is published, and the calibration is named as ours.** The scale anchoring

is this product's calibration, not the test maker's, and the page that explains the method says so and stays in step with the code.

Two further rules are specific but generalise:

- **Never mix incomparable scales.** Two editions of one exam use different ranges and are

not convertible; they are compared by percentile only, with the concordance cited. Any product with a legacy unit and a current one has this problem.

- **Never present an internal threshold as official.** A routing cut chosen here is ours

and is labelled ours.

The pattern underneath all of it: **when you show someone a number about themselves, the number is a claim, and the claim has an error bar whether or not you print it.** Printing it is the difference between a product people trust and one they catch out.

Content at Scale, Without Lying About It

This platform ships over a hundred thousand practice items across 54 bank files. Almost all are generated. The chapter is about how to do that without producing a number that is technically true and substantively false.

Generators, not text

An item is produced by a schema: a template plus a parameter space plus a rule for generating distractors. The schema knows how to make a correct answer and how to make wrong answers that are wrong for a reason.

Every key is computed, never asserted. The generator does not record which option is correct; it computes the answer from the parameters and then places it. An asserted key is a key that can be wrong, and a wrong key in a practice bank is the worst possible defect because the learner concludes they are wrong.

The counting problem

The hard part of generated content is not generating it. It is counting it honestly. This project got the count wrong in four distinct ways.

The dedup key must be canonical under every transformation the item legitimately undergoes. Answer position is randomised per draw, and the key hashed choices in order, so one question with its options rearranged counted as two. Sort before hashing.

Identity is not always the choices. For reading items, identity is the passage plus the question asked. Four passages produced five hundred "distinct" stated-idea items that differed only in which of the passage's other sentences came along as distractors. A student who has answered one has answered all of them.

An over-broad key deletes as silently as a narrow one inflates. Two different conditional questions produced the same stem, hashed identically, and half the inference items in any corpus would have vanished without a word.

A filter that drops must say so. A length guard measuring the wrong unit silently removed sixteen valid items. Every rejection stage reports its rejection count.

The honest yield number for reading items here, once both dedup bugs were fixed, is about six stated-idea and two inferred-idea questions per passage. That means five hundred inference items needs

roughly a hundred and seventy passages. **That number is the entire cost of the category, and it is recorded rather than discovered later.**

Quality ratchets

A generated bank drifts toward exploitable patterns unless something measures them. Two that mattered here:

The length tell. In one section the longest option was the correct answer 81 percent of the time. A test-wise student does not need to read the question. The generator now draws distractors so that the key's length rank is uniform, and a ratchet in the test suite fails if the tell reappears. It is down to chance.

Comparable development. Each added clause in a distractor is chosen to leave the option wrong for the reason it was already wrong, so length and elaboration carry no signal about correctness.

A ratchet is a pinned number that may only move one way. If a fix makes a statistic better, pin the better number. This is the cheapest possible regression test for anything statistical, and it works where an exact assertion cannot.

Shipping a large bank to a browser

At a hundred thousand items the bank is tens of megabytes, and this is where content scale becomes a performance problem.

- The whole bank as a blocking script meant **20.3 seconds to the first question on

throttled 3G**. Nobody noticed, because nobody had measured it.

- The fix is a strided starter bank, small and covering every category, plus an async

remainder that pushes into the same array. Time to first question fell to 7.6 and 11.0 seconds, and blocking download from 11.3 MB to 1.2 MB.

- Platform limits bite here. Cloudflare rejects a single static asset over 25 MiB, so the

remainder is chunked, and the build fails on any asset over the limit.

- When you change a file naming scheme, grep for the old name as a **string**, including

inside regular expressions. A service worker precache pattern of `bank_rest\.js$` does not match `bank_rest1.js`, and the failure is silent degradation of offline support.

The sourcing discipline for facts

Generated practice items are one thing. Published facts about the world are another, and they need a different regime.

- **Every figure carries a source, a year and a URL.** Not a citation in prose: fields, on

the record, validated at build time.

- **Unverifiable means null, and null renders as a dash.** Never a guess, never a plausible round number.

- **A named list of banned sources**, enforced by the validator. Forums, aggregators, crowd-edited encyclopaedias and coaching-company blogs. A build fails if one appears.

- **One file per entity**, so a fix touches one file and a validator can walk them all.

This project holds 94 school files under that scheme.

- **When a source cannot be reached, say so immediately and stop.** Two exams on this site

are blocked, and recorded in the roadmap as blocked, because the test makers' pages return bot challenges. The rows that exist for them cite a banned source and are marked for replacement rather than quietly reused. Being blocked and saying so is a better state than being unblocked by a guess.

The Data Layer

16 tables across 31 migrations, with the database rather than the application as the enforcement point.

Row Level Security as the default posture

1. **Enable RLS on every table at creation.** Not in a later pass.
2. **No policy means no access**, which is exactly right for anything only your server

should touch. This project's telemetry, error and billing tables all have RLS on and no policies at all; the service role bypasses RLS, and admins read through functions.

1. **Public read where public read is the point**, scoped: forum rows that are not hidden, for instance.

1. **Open insert with server-side stamping.** The forum takes posts from anyone with no account. The client cannot set the author, the country or the address hash; triggers do. A client-supplied field is a claim.

The **SECURITY DEFINER** pattern

Every privileged read is a function that checks admin membership and then reads. Clients never touch the underlying table.

Two rules that are easy to get wrong:

Revoke from PUBLIC, not just from anon and authenticated. A leading `=X/postgres` in `pg_proc.proacl` means `PUBLIC` holds `EXECUTE`, and revoking from every role you can name changes nothing. Verify with the platform's advisors or by reading the ACL. Never verify by reading your own migration.

Never take an id from the request body. The cancel-subscription and portal functions here read the caller's id from their own RLS-scoped row. A function that accepts an id and acts on it is a function that acts on anyone's row.

Triggers, and the two that bit

Postgres fires same-timing triggers in alphabetical order by name. On this project `profiles_sharing_eligibility` sorted before `profiles_sync_age_tier`, so eligibility was judged

against a stale age tier and **silently revoked every legitimate adult opt-in**. If two triggers on one table have an order dependency, encode it in the name with a numeric prefix.

UPDATE OF <columns> fires on the columns named in the statement, not on what actually changed, and not on what an earlier `BEFORE` trigger wrote into `NEW`. So `update profiles set birth_year = 2011` ran the sync, stored a minor's tier, and skipped the eligibility check entirely. If you need "when this value changed", fire on every update and compare `OLD` to `NEW` yourself.

Both were found by a SQL smoke test that asserted the **guarantee** and rolled itself back. Neither would have been found by reading the trigger.

Ordering, idempotency and the event problem

Webhooks do not arrive in order. Notifications do not arrive in order. This is not an edge case, it is the normal condition, and the consequences are severe: a `deleted` landing after an `updated` resurrects a cancelled plan, and the reverse downgrades a paying customer.

The discipline, which applies to any external event source:

1. **Never write the event payload.** The payload tells you *which* thing changed. Re-read

the current state from the source of truth and write that.

1. **Guard on a monotonic field.** A notification older than what you last wrote is

discarded. Apple provides `signedDate`; most systems provide something.

1. **Terminal states are terminal.** A refund or a revocation is never overwritten by a

late renewal.

1. **Put idempotency in the database, not the handler.** A unique index on the natural key

is correct under concurrency. A check-then-insert in application code is not.

Retention

- Unlinkable telemetry: 400 days here, on a scheduled job rather than a trigger, because

a delete on every insert puts a table scan in the path of the user's action.

- The retention window should match the longest window your console can query, so nothing

the interface can ask for is ever missing.

- **A billing ledger is not telemetry.** It is a financial record and it is kept.

Migrations as a readable record

Every migration in this repository opens with a comment explaining what it does and, more importantly, **why the obvious alternative was rejected**. Read six months later, the "why not" is the part you need; the "what" is in the SQL.

Keep the applied migrations mirrored in the repository, and keep a manifest with checksums. Applying through a console and forgetting to mirror is how a schema and its history diverge, and the divergence is invisible until somebody tries to rebuild.

Privacy as a data-model decision

Two ideas here are worth copying wholesale.

Unlinkability by construction. The item telemetry table has no user column, no session column, no device column and no address column, **by design, and must never gain one**. That absence is precisely why it needs no consent gate. Privacy enforced by a schema is stronger than privacy enforced by a policy document, because a schema cannot be forgotten.

Keep what someone told you separate from what you were told about them. Self-reported attributes live on the profile. Third-party appended attributes live in a separate table that cascades on deletion. They are never merged, so what a person said about themselves stays distinguishable from what was bought about them, forever.

Payments

5 edge functions handle money here: `apple-notifications`, `cancel-subscription`, `create-checkout-session`, `create-portal-session`, `stripe-webhook`.

The shape

Hosted checkout, not a card form. The processor's hosted page removes PCI scope entirely, and there is no version of building your own that is worth the saving.

The webhook is authenticated by the processor's signature rather than by your own auth, which means it must be deployed with JWT verification **off** and the signature check **on**. Get that pairing wrong in either direction and you have either an endpoint that rejects every legitimate call or one that accepts every illegitimate one.

Every column the webhook writes is service-role only, left out of the column grants entirely, so no browser session can forge a plan. **A signed-in user was once able to grant themselves a paid plan on this project**; that is what the grant hardening is for.

Two processors, and why the plan column stops being the answer

The moment a second payment source exists, "what has this person paid for" cannot be a column, because it becomes a race: whichever webhook fired last wins, and a user who subscribed on iOS gets downgraded the next time a card event lands on their row.

The resolution here: the original column stays exactly as it was, owned by the original processor and untouched by anything new, and a function derives the **effective entitlement** across every source. The application reads the function. Nothing about the live, money-taking path changed, which was deliberate.

Generalisation: when a second source of truth appears, do not teach the first one to share. Add a derivation on top and move readers to it.

Promises that are load-bearing

Write these down and test them, because they are the ones that turn into complaints.

- **The free plan never asks for a card.** Tested in both signed-in and signed-out states.
- **Cancelling is one click, in the account page, through your own function.** Not through

the processor's hosted portal, which may not be configured, and which puts the most important retention moment on somebody else's page.

- **The exception you cannot engineer around.** An Apple-sourced subscription is Apple's:

there is no API to cancel on a user's behalf and the guidelines require pointing at Settings. So the button opens Apple's subscription management and explains why, rather than appearing to work and not.

Which path applies is decided by the entitlement record, never guessed from the platform the page happens to be open on.

- **A refund window, stated, plus your own errors always refunded.**
- **Access continues to the end of the paid period.**

The ledger, and the mistake to avoid

The most consequential payments decision in this project was made late and should have been made first.

The profile row holds what an account pays **right now**, and it is overwritten on every webhook. That means the instant a subscriber cancels, the evidence they ever paid is gone. It is enough to gate access and useless for running a business: it cannot answer how much revenue arrived this month, how much left, what share of trials converted, or what came back out as refunds.

Build the append-only billing event ledger before the first real subscriber. It cannot be reconstructed afterwards.

The design that works:

- One row per fact, normalised across processors so a chart reads one vocabulary.
- **Cash and recurring value in separate columns.** An annual plan is \$99.99 of cash on one

day and \$8.33 of monthly revenue. A dashboard that adds those together is wrong in both directions.

- A `kind` column of your own verbs, not the processor's event names.
- `mrr_delta_cents` on every row, so new plus expansion minus contraction minus churn is

the movement bridge by construction, and cannot be off by a rounding rule applied in one place and not another.

Four traps, all of which would have produced a plausible wrong number:

Two event types announce one new subscription. Hosted checkout completion and subscription creation both fire, with different event ids, in no guaranteed order. Logging both doubles new revenue. A

partial unique index on (source, subscription id, kind) for the once-per-lifetime kinds lets the database decide which arrived first.

The first invoice is also an invoice. Counting every paid invoice as a renewal double-counts every new subscriber as a returning one. Only a cycle invoice is a renewal. The cash still counts.

The refunded amount is cumulative. A charge reports total refunded to date, not the amount of this refund, so a second partial refund counts the first one again. Use the most recent refund entry.

The cancellation write erases what you need to record the cancellation. Read the prior state before writing, or churned revenue is always zero.

Sandbox is not money. Test transactions carry real-looking prices. A sandbox row in a revenue chart is a fictional dollar, and it will be believed.

Know the units. One processor reports price in milliunits: 4990 means \$4.99. Reading it as cents overstates revenue by a factor of ten, which is an error that looks like success.

Secrets

Secret keys and signing secrets live in the platform's secret store. Never in the repository, never in an environment file that is committed, never in a log line. This is the one rule in this book with no nuance attached to it.

Business Intelligence

The three streams

Keep them separate and know what each can answer.

Site events. A first-party beacon on every page: path, referrer, UTM, a session id from `sessionStorage`, device class, duration. Country and a salted address hash added by a trigger so the raw address is never stored. Behind a consent banner that honours Global Privacy Control and can be reopened from any page footer.

Product telemetry. What happened inside the product. Here that is item telemetry with **no user, session, device or address column, by design**. It answers "is this item too hard" and cannot answer "what did this person do", and that is the trade that lets it run without a consent gate.

Billing events. Money. Covered in the payments chapter.

They are deliberately not joined. Nothing here attributes a subscription to a visit. That is a design choice with a real cost, and it should be stated on the page rather than quietly left as a gap the reader assumes is filled.

Read through functions, always

No client reads an analytics table. Every admin view is a `SECURITY DEFINER` function gated on admin membership, returning exactly the shape the page needs.

One read per page, not six. Six reads can disagree with each other: the tile says eleven paying and the chart says twelve because a webhook landed between the two queries. One statement, one snapshot, one story.

The numbers not to compute

This is the part that separates a dashboard from a decoration.

A churn rate needs a denominator held at the start of the window. If your ledger is younger than the window, you cannot supply one. The counts and the lost revenue are real; the ratio would be a guess wearing a percentage sign. This dashboard prints the counts and explicitly declines the rate, and says why, and starts printing it automatically once the ledger is old enough.

A conversion rate must exclude the undecided. A trial still running is not a failed trial. Counting it in the denominator produces a rate that rises on its own as trials mature, which is worse than no rate at all.

Trial revenue is never revenue. State the pipeline figure separately and label it as conditional.

Gross and net are different numbers. Show refunds beside charges, not subtracted from them. Netting hides the month where a fifth of revenue came back.

A snapshot take rate is not a cohort conversion rate. Say which one you are showing.

Say what the data covers

Every derived figure is "since the log began". Print that date on the page. A figure that silently covers less than its label claims is the hardest kind of wrong to catch, and it is the failure the whole discipline exists to prevent.

The triage loop

Errors and moderation both work the same way, and the pattern is worth naming.

1. Capture into an insert-only table.
2. Cluster by signature, and attach the evidence to the cluster: the real message, the

real stack, the builds, the pages, how many separate sessions hit it.

1. **A human decides.** Real, fixed, will not fix, or noise.
2. The decision is stored, and it mutes the cluster permanently.

The queue gets quieter as it learns rather than louder, and the stored labels are exactly what a future model would train on. Nothing in it edits code, and that is deliberate: trust is the binding constraint on automated repair, so the system proposes and explains, and a person decides.

Never fabricate

No invented user counts, testimonials, efficacy claims or seeded activity. On a forum, house-authored posts say they are house-authored. This is not only an ethics rule; a fabricated number in a dashboard eventually gets used for a decision, and by then nobody remembers it was a placeholder.

Tests and Guards

21 test files, and the interesting thing about them is not what they assert. It is that the analysis chapter can count how defects were **actually** found, and the answer reshapes where you put effort.

The ladder

Rung 1: the build. Structural guards on the artefact. Parse every generated script. Assert every collection size. Fail on platform limits. Compare inputs to outputs at every filter. Cheapest rung, catches the most.

Rung 2: domain logic, headless. The engine, run without a browser, once per variant. Fast enough to run on every change.

Rung 3: the real browser against the real built site. Playwright against the built output over a local server, at desktop and phone width, in both themes. **Assert behaviour, not markup.** A markup assertion breaks on every refactor and passes through every real bug.

Rung 4: adversarial play. A bot that uses the product the way a user would, repeatedly, and reports what only use reveals. Three real defects here came from nowhere else.

Rung 5: the periodic audit. Contrast, links and metadata across every page, weekly. Catches the slow rot that no single change is responsible for.

The failure modes of tests themselves

Five entries in the defect ledger are about tests being wrong, and that proportion is not an accident. **A broken test is invisible, because its output is identical to a working one.**

A negative assertion passes when the system is broken in the right way. "Nothing is sent when consent is refused" passed here because the test used the wrong storage key, so consent was never configured, so nothing was ever sent. It would have passed with the feature deleted. **Always pair a negative assertion with the positive case**, or you are asserting nothing.

A test can measure the wrong instant. A performance test waited for the browser load event, which waits for the async resource the optimisation moved off the critical path. A change that made the page usable nine times faster would have reported no improvement. **Measure the moment the user can act.**

A guard can narrow its own scope. A count check matching `\d{2,4}` stopped checking the two largest counts when they passed ten thousand, and kept passing. **Assert how many things you checked**, not only that the checks passed.

A test can measure something that was never rendered. A hidden element still answers `innerText` and `getComputedStyle`. Layout and contrast assertions passed here on a view that was never displayed. **Assert the thing is on screen before you measure its layout.**

A test can classify by name instead of by value. A theme parity check selected tokens by name prefix and flagged a font size as a missing colour.

Ratchets

For anything statistical, pin the number and allow it to move one way. If a fix improves a statistic, pin the better value. This is how the length tell in the item bank went from 81 percent to chance and stayed there.

A ratchet only works if it is stable. This project's ratchets drifted run to run because the generator seeded from a randomised hash, and a flaky ratchet gets deleted. **Seed from something stable and assert reproducibility first.**

Write the expected number before the code

The dashboard test here carries a fixture of two subscriptions and a comment block computing, by hand, what every tile must show. The test knew \$159.84 was the right answer before the page existed, which is how it caught the page printing \$160.

If you cannot write down the expected output before you write the code, you do not yet understand the requirement.

Wire it into CI, and prove it ran

The browser suites on this project **had never actually run**. Playwright was a declared dependency in a repository where nothing had installed it. The tests existed, the command existed, and the coverage was zero.

A test that is not wired into CI is a test that does not exist. Prove a suite runs in the place it is supposed to run.

Render it and look at it

Three defects here were found by taking a screenshot: a chart unreadable in dark mode, a line chart implying values that never existed, and a dashboard that was not on screen. None of them would have

been caught by an assertion, because none of them were violations of anything anybody had thought to assert.

Budget for looking. It is the cheapest test there is and the only one that catches things you have not imagined.

Running the Build as an AI Loop

75 commits in 36 days, one owner, a series of AI sessions. This chapter is how that was actually run, including the parts that did not work.

The division of labour

The owner decides: what to build, what it means, what is acceptable, and every irreversible or outward-facing action. **The session does:** the building, the measuring, the testing, and the reporting of what it found.

The failure mode at each end is worth naming. An owner who reviews every line becomes the bottleneck and the throughput collapses to the speed of reading. A session that decides product questions on its own produces something coherent that is not what anyone wanted.

The rules file is the highest-leverage artefact

One file, read every turn, holding only load-bearing rules. Not documentation, not architecture notes. The test for whether a line belongs in it: **would you otherwise have to say this again?**

What earns a place:

- Style rules a build can enforce.
- Sourcing rules, with a named list of banned sources.
- **Never invent a value.** Every id, hash, count, price, date and URL must be read from

real output before it goes into a tool call, a commit or a page.

- **Report a blocked resource immediately** and ask, rather than quietly working around it.
- The handful of architectural facts that are expensive to rediscover.

Every one of these was written after something went wrong. Write them at the start anyway; the list above is transferable as-is.

Ratchets, not reviews

The owner cannot read every line. So the correctness that matters is encoded in things that fail loudly: build guards, pinned statistics, browser suites, review bots. The review budget then goes to the few decisions that are genuinely judgement, rather than being spread thinly over everything.

This is the same principle as the data layer. **Push the guarantee down to the layer that cannot be bypassed.**

What a session should report

Not a list of files changed. The useful report is:

1. What was built, in one paragraph.
2. **What broke on the way, and why.** This is the most valuable part and the easiest to

omit, because it reads as failure. It is the opposite.

1. What is now guarded against, and where.
2. What was left undone, and whether that is a decision or a blocker.
3. Numbers, with the command that produced them.

The commit messages in this repository are written that way, which is why a defect ledger could be reconstructed from them 36 days later. **Write the commit message as though someone will need to mine it. Someone will.**

Failure modes observed in this project

Reporting a regression from one seed. A stochastic measurement compared before and after on a single seed showed an 88 to 72 percent drop. Across three seeds the spread between seeds was larger than the effect. Run more seeds before reporting.

Pausing to report when told to keep working. A session that stops to summarise every few minutes converts working time into reading time. If the owner has said to keep going, keep going, and batch the reporting.

Treating a policy document as a blocker. A missing stylesheet reference was reported as a privacy concern because the file lived in a directory named after the privacy page. Be precise about what kind of problem you have found; a 404 is a 404.

Typing a value that looked right. A forty-character hex string was typed into a merge call as though it were a commit SHA. It was rejected as malformed, which was luck; a well-formed guess would have merged the wrong thing. `git rev-parse` costs one call.

Adding a guard that did not guard. Several ledger entries are guards that were too narrow, too broad, or measuring the wrong thing. After writing a guard, break the thing on purpose and confirm the guard fails.

The cadence that worked

Long uninterrupted sessions with standing authorisation, punctuated by the owner redirecting priorities, beat short supervised ones by a wide margin. The standing authorisation that made it work was specific: merge your own work once the checks are green, do not check in on each item, and stop immediately when told to.

The thing that makes that safe is not trust. It is that the checks are real, the destructive actions still require asking, and the record of what happened is written down well enough to audit later. Which is what this book is.

What the Ledger Says About Itself

83 recorded defects, over 36 days of building. This chapter is computed from the ledger every time the document is built, so it cannot fall out of step with it.

How defects were actually found

HOW	COUNT	SHARE
Found by reading the code or the output	36	43%
Found by measuring something	22	27%
A test caught it	13	16%
Found by rendering it and looking	5	6%
Found by a review bot or an adversarial pass	5	6%
A person hit it	1	1%
A build guard caught it	1	1%

This is the most useful table in the book. 82 of 83 defects, 99 percent, were caught by something other than a person hitting them in production. The single largest category is not a clever tool: it is reading the built output instead of the source that produced it. The second is measuring a number nobody had measured before. Neither requires infrastructure, and both are habits rather than tools.

Read that percentage with the bias it carries. This ledger is written by the people who found the defects, so it counts what was caught and cannot count what was not. A defect a user hit and nobody recorded does not appear here. The honest reading is not "97 percent of all defects were caught early"; it is "of the defects we know about, almost all surfaced through one of these five habits", which is still the useful claim, because it says where to spend attention.

By severity

SEVERITY	COUNT
Wrong data shown or stored	33
Silent loss	19
Degraded	17
Cosmetic	11
Site down	3

Silent loss is the dominant failure mode, at 19 of 83. Not a crash, not an error page: something quietly did less than it claimed. A loop over an empty list, a filter that dropped rows, a guard that stopped checking, a table that never received a write. None of these announce themselves, and none are caught by error monitoring, which is why the guard ladder in this book is built around asserting counts rather than catching exceptions.

By area

AREA	COUNT
Content generation	23
Tests and guards	17
Front end	8
Build system	6
CSS and layout	5
Payments	5
Infrastructure and deploy	5
Scoring and selection	4
Database	4
Search and metadata	3
Interface and data display	3

Guard coverage

77 of 83 defects produced an automated guard. 6 did not, and are carried by attention alone, which means they are the ones most likely to recur.

Carried by attention:

- **INC-0010** CSS comments do not nest, and one placeholder killed the palette on 1451 pages
- **INC-0019** The audit reported a working link as broken
- **INC-0025** A stale edge cache made a fixed 404 look like a live 200
- **INC-0034** A line chart interpolated between discrete money events
- **INC-0036** charge.amount_refunded is cumulative, so partial refunds double-count
- **INC-0045** The error reporter reported its own failures, in a loop

Guards that did not hold

The same guard named by two incidents is a guard that did not hold the first time. These are the places to spend effort.

- `weekly_audit` checks computed colour on every page (INC-0050, INC-0048)

Where defects concentrate

Files named by three or more incidents. This is not the same signal as the list above: a file that is the natural home for many checks will appear here without any one of them having failed. It says where the work has been, and where a reader new to the codebase should look first.

- `src/build.py` , 11 incidents (INC-0001, INC-0002, INC-0017, INC-0027, INC-0059, INC-0060, INC-0063, INC-0064, INC-0067, INC-0076, INC-0080)
- `src/build_banks.py` , 7 incidents (INC-0003, INC-0007, INC-0008, INC-0009, INC-0011, INC-0079, INC-0081)
- `src/test.js` , 7 incidents (INC-0004, INC-0038, INC-0039, INC-0040, INC-0043, INC-0044, INC-0069)
- `src/review_bot.js` , 5 incidents (INC-0022, INC-0026, INC-0051, INC-0061, INC-0077)
- `src/bank_emit.py` , 4 incidents (INC-0062, INC-0066, INC-0068, INC-0073)
- `src/weekly_audit.js` , 3 incidents (INC-0050, INC-0048, INC-0018)
- `src/smoke_redirect.js` , 3 incidents (INC-0023, INC-0024, INC-0047)
- `src/build_playbook.py` , 3 incidents (INC-0057, INC-0065, INC-0083)
- `src/bank_repair.py` , 3 incidents (INC-0070, INC-0071, INC-0072)
- `src/gen/framework.py` , 3 incidents (INC-0074, INC-0075, INC-0078)

The Defect Ledger

Every entry here happened. Each one is a record of something that broke, how it was found, what fixed it, and what now stops it coming back. The last field is the one that matters for a different business: the transferable lesson, stated without reference to this codebase.

They are grouped by the part of the system, and within a group by date. The `guard` field feeds the checklist chapter automatically, so nothing here has to be copied anywhere by hand.

Content generation (23)

INC-0003. Item banks were different on every build because Python randomises hash()

2026-09-16, *Wrong data shown or stored*, `0d6c357` PR #33

- **What was seen.** Two builds of the same commit produced different banks, and the per-section bias figures pinned in the test file drifted run to run.
- **Why.** `build_banks.py` seeded each category with `abs(hash(exam + skill))`. Python randomises string hashing per process by default.
- **How it surfaced.** A pinned test figure would not stay pinned. (A test caught it)
- **Fix.** Seed from `zlib.crc32`, which is stable across processes.
- **What stops it now.** three consecutive builds must produce identical output in `src/build_banks.py`
- **Cost.** a flaky ratchet that would have been disabled eventually
- **Lesson.** Any generator that claims reproducibility must be seeded from something stable across processes. `hash()` is not, in Python, and the failure shows up as a flaky test rather than as a wrong answer.

INC-0004. A shadowed variable silently deleted 3000 items

2026-09-16, *Silent loss*, `aecccd9` PR #37

- **What was seen.** ACT Maths dropped from 4500 items to 1500 and the build reported success.
- **Why.** Adding an ACT English mapping declared a second `ACT_MAP` that shadowed the existing one. Nothing errors when a map is smaller than it used to be.
- **How it surfaced.** Comparing the reported category counts against the previous build. (Found by measuring something)
- **Fix.** Merge the maps properly and read the section per skill.
- **What stops it now.** the build prints per-category counts and the test file pins them in `src/test.js`
- **Cost.** 3000 items, caught before merge

- **Lesson.** Deletion by shadowing is invisible. Any collection whose size is a fact about the product needs its size asserted, not just its contents.

INC-0039. 225 of 302 correct answers sat at position A

2026-09-16, Wrong data shown or stored, 81eecec PR #30

- **What was seen.** Three quarters of the correct answers in a new bank were the first option.
- **Why.** Items were authored with the key written first and the position never randomised.
- **How it surfaced.** Counting the key positions, a check nobody had run. (Found by measuring something)
- **Fix.** Randomise the key position per draw.
- **What stops it now.** the test suite pins the key-position distribution in `src/test.js`
- **Cost.** a bank that could be beaten without reading it
- **Lesson.** In any set of multiple-choice content, count where the answers are. A positional tell makes the whole set worthless to a test-wise user, and it is invisible item by item.

INC-0044. The longest option was the correct answer 81 percent of the time

2026-09-16, Wrong data shown or stored, 425a8bb PR #35

- **What was seen.** A student who picked the longest option without reading the question scored 81 percent against a chance level of 20.
- **Why.** Distractors were shorter than keys because a correct answer is naturally more qualified, and nothing measured the resulting length distribution.
- **How it surfaced.** Measuring the accuracy of a strategy that ignores the question entirely. (Found by measuring something)
- **Fix.** About ninety items had distractors rewritten to carry comparable development, each added clause chosen to leave the option wrong for the reason it was already wrong. 81 percent to 38, then to chance in a later pass.
- **What stops it now.** a ratchet pins the length tell and fails if it rises in `src/test.js`
- **Cost.** a section that could be beaten without reading it
- **Lesson.** Test your content against the strategies a lazy adversary would use, not only against whether it is correct. Measure the score of a rule that ignores the question.

INC-0007. The dedup key counted a reshuffled question as a new one

2026-09-17, Wrong data shown or stored, fc4114c PR #39

- **What was seen.** Bank counts were inflated. The same question with its options rearranged looked distinct.
- **Why.** `canon()` hashed the choices in the order they appeared, while the answer's position is randomised per draw.
- **How it surfaced.** Sorting the choices before hashing, which immediately collapsed the counts. (Found by reading the code or the output)
- **Fix.** Sort the choices inside `canon()`.
- **What stops it now.** `canon()` sorts, and the build reports distinct counts per category in `src/build_banks.py`
- **Cost.** four ACT categories reported far above their real size
- **Lesson.** A deduplication key must be canonical under every transformation the item legitimately undergoes. Ask what varies per draw before you hash.

INC-0008. Four passages produced five hundred fake distinct questions

2026-09-17, *Wrong data shown or stored*, `61c3ed2` PR #40

- **What was seen.** A reading generator with four passages reported five hundred distinct stated-idea items.
- **Why.** The identity of a reading item is the passage plus the question asked, not the choices offered. The same stem with the same key varied only in which distractors came along.
- **How it surfaced.** Reading the generated output rather than its count. (Found by reading the code or the output)
- **Fix.** `canon()` lets a generator declare that its identity is the passage plus the question.
- **What stops it now.** per-category distinct counts printed at build time in `src/build_banks.py`
- **Cost.** would have shipped a bank five hundred deep and four questions wide
- **Lesson.** The same inflation arrives through a different door every time you close one. When you fix a dedup bug, ask what else shares an identity.

INC-0009. Two conditionals hashed identically and half the inference items would have vanished

2026-09-17, *Silent loss*, `61c3ed2` PR #40

- **What was seen.** Half of a passage's inference questions disappeared from the corpus.
- **Why.** Both conditionals produced the stem 'which of the following can be properly inferred from the passage', so they hashed to the same value and one was dropped as a duplicate.
- **How it surfaced.** Measuring true yield per passage. (Found by measuring something)
- **Fix.** Each conditional names the case it asks about.

- **What stops it now.** yield per passage is measured and recorded, not assumed in `src/build_banks.py`
- **Cost.** half the inference items in any corpus
- **Lesson.** Dedup can be wrong in both directions. An over-broad key deletes real content as silently as a narrow one inflates it.

INC-0011. A length guard silently dropped sixteen valid items

2026-09-17, *Silent loss*, `61c3ed2` PR #40

- **What was seen.** Sixteen items never appeared in the bank.
- **Why.** The guard measured characters, and the items were legitimately longer than the limit in characters while being normal in content.
- **How it surfaced.** Counting inputs against outputs. (Found by measuring something)
- **Fix.** Measure what the guard actually cares about.
- **What stops it now.** input and output counts are compared at every filter stage in `src/build_banks.py`
- **Cost.** sixteen items
- **Lesson.** Every filter needs its rejection count reported. A filter that silently drops is indistinguishable from an input that was never there.

INC-0062. Correcting a length tell moves it one rank over, every time

2026-09-22, *Degraded*, PR #66

- **What was seen.** On two separate hand written banks, extending one distractor per item took the longest-is-key rate to near zero and left 25 of 35 and then 30 of 42 keys sitting second longest, so a reader picking the second longest option scored 71 percent on both.
- **Why.** Mechanical rather than careless. Extending exactly one distractor past the key moves every key from rank 5 to rank 4 by construction. The recorded ratchet measures only the two extremes, so a corrected bank passes it while carrying a stronger tell one position in.
- **How it surfaced.** Measuring the full length rank rather than only the extremes, after the extreme came back clean. (Found by measuring something)
- **Fix.** Three passes rather than one, extending a second and third distractor on overlapping subsets, and a shared `measure()` that prints the whole rank distribution and the best single-rank strategy so the artefact cannot hide behind a passing extreme.
- **What stops it now.** `bank_emit.measure` prints the full rank and the best single-rank score in `src/bank_emit.py`
- **Cost.** two banks that would have passed the ratchet carrying a 71 percent tell

- **Lesson.** A guard on the extreme of a distribution can be satisfied by moving the mass next to the extreme. When you correct for a measured bias, measure the whole distribution afterwards, not the statistic you were correcting.

INC-0066. Half the length tell correction did nothing and the build said it had worked

2026-09-22, Silent loss

- **What was seen.** The new GMAT reading bank corrected its length tell with a table that carries a named number of distractors past the key on each item, so the keys land at spread ranks rather than all at one. The table named 65 items. On 31 of them the authored clause was shorter than the gap it had to close, so the distractor stayed below the key and the item did not move. The run printed an improved distribution and no error, and the 31 were only found by measuring intent against outcome by hand.
- **Why.** `extend()` appends whatever clause the author supplies and checks only that the needle matches exactly one non key choice. Whether the choice ends up longer than the key, which is the entire point of the call, was never checked. The author sizes each clause by eye against a gap reported in a separate run, and an estimate made that way is wrong about half the time.
- **How it surfaced.** Comparing the intended lift count for each listed item against the key's actual rank after the run, in a scratch script written because the printed distribution was flatter than before but not as flat as the table should have made it. (Found by measuring something)
- **Fix.** `check_lift()` in `bank_emit.py` takes the same intent the tables express, one entry per item saying how many distractors were meant to pass the key, and exits naming every item whose key did not land at the matching rank and by how much the clause fell short. The generator calls it after the last `extend` pass, so a clause that does nothing fails the build instead of being averaged into a number that looks better.
- **What stops it now.** `check_lift` asserts each lifted item's key rank matches the number of distractors the table lifted in `src/bank_emit.py`
- **Lesson.** A correction table is a set of claims about outcomes, and an entry that quietly fails still counts as applied. Aggregate metrics hide this well: a table where half the entries work still moves the number in the right direction, which reads as success. State the per item intent in a form the machine can check, and every entry that did nothing says so by name.

INC-0068. Shuffling the answers twice put 37 percent of the keys at A

2026-09-22, Silent loss

- **What was seen.** The new LSAT reading generator printed a key position distribution of 33, 15, 14, 17, 11 across 90 items where an even split is 18 each. Position A held 37 percent of the keys, against a chance rate of 20, which is the exact defect `permute()` exists to prevent (INC-0039) and which the engine test's answer position check would have failed.

- **Why.** The generator called `E.permute()` twice. An intermediate run had ended with `permute` and `measure` so the skill counts could be read, and the final block added its own `permute` before the `extend` pass, and the earlier pair was never removed. `permute` seeds a shuffle with `crc32` of the item `id`, so the second call applies the same permutation again, composing it with itself. A permutation composed with itself is not uniform: it favours its own fixed points and short cycles, and with five choices that piles keys onto the position they started at, which is `A` because items are authored key first.
- **How it surfaced.** Reading the generator's own `measure` output after the `length` tell was cleared. The `length` rank line was flat and the key position line on the same screen was not, and it had been on the screen for several runs before it was read. (Found by measuring something)
- **Fix.** `permute()` now stamps each item it has shuffled and exits if it is handed one twice, naming the item. The duplicate call in `mk_bank_lsatsrc3.py` is removed. The measured distribution after the fix is 20, 19, 19, 16, 16 across 90 items, and the `length` rank is 18 in every position.
- **What stops it now.** `permute` refuses to shuffle an item it has already shuffled in `src/bank_emit.py`
- **Lesson.** A seeded shuffle is deterministic, which makes calling it twice look harmless: the same input gives the same output. What repeats is the permutation, not the randomisation, and a permutation applied to its own result is biased toward leaving things where they were. Any function whose value comes from being applied exactly once should refuse to be applied twice rather than relying on the caller to remember. The second lesson is about reading: the generator printed the defect on every run, above the line being watched.

INC-0069. A bank a student can play at 88 percent, inside a section the check passed

2026-09-22, Degraded

- **What was seen.** `bank_sat_rw.js` ships 32 items on which the longest answer choice is the key 88 percent of the time, and a student who always picks the longest option scores 84 percent on that file against a chance rate of 25. Eighteen other hand written bank files across all five exams carry the same tell at lower strength, several above 50 percent. The engine test reported the SAT Reading and Writing section at 35 percent and passed it.
- **Why.** The `length` bias check groups items by section. A section mixes hand written items with generated ones, and the generated items are flat by construction because the schemas assemble choices mechanically. For SAT Reading and Writing the generated bank supplies 160 of 348 items at 7 percent, which pulls a hand written 88 percent down to a section figure of 35 and under the recorded tolerance of 36. The aggregate was not wrong; it was answering a question nobody needed the answer to. A student does not meet a section, they meet items, and the items arrive from one file at a time.
- **How it surfaced.** Measuring the same statistic per source file rather than per section, which was written to find out where the new LSAT and GMAT banks sat relative to the old ones and reported the old ones instead. (Found by measuring something)

- **Fix.** The check now runs per hand written source file as well as per section, with its own recorded baseline per file, so a file can only improve. Numeric answer banks are excluded because the separate numeric rank check covers them and character length is the wrong measure for a number. `src/bank_repair.py` applies an authored clause table to a hand written bank in place, by textual substitution inside the named choice, so the remaining files can be corrected without regenerating a file that was never generated.
- **What stops it now.** length bias is measured per hand written bank file against a recorded baseline, not only per section in `src/test.js`
- **Lesson.** An aggregate over a mixed population reports the population, and if part of that population is flat by construction it will hide the part that is not. The rule that follows is about what the unit of the measurement should be: measure at the grain the defect can exist at, which here is the file, because a file is written by one person in one sitting with one set of habits. The section was the grain the data was convenient at.

INC-0070. The second tool that appends a clause did not carry the first one's rule about the full stop

2026-09-22, Cosmetic

- **What was seen.** Two repaired SAT items read 'the number of purchases. made' and 'sleep debt over the course of a week. that they do not fully repay at the weekend'. The clause had been appended after the sentence's own full stop.
- **Why.** `bank_emit.extend` has handled this since it was written: it strips a trailing period, appends the clause and puts the period back. `bank_repair.repair` was written months later for hand written banks that a generator never produced, and it inserts at the closing quote of the string literal, which is after the period. The rule lived in one implementation rather than in a place both could use, and the second implementation was written from the problem rather than from the first solution.
- **How it surfaced.** Reading the per item report after the repair, where the appended text appears at the end of each changed choice. (Found by reading the code or the output)
- **Fix.** `repair()` now inserts before a trailing period, matching `extend()`. Both behaviours are stated in one comment naming the other function, so the next tool that appends to a choice is written from a rule rather than from scratch.
- **What stops it now.** `bank_repair.repair` inserts before a trailing full stop, as `bank_emit.extend` does in `src/bank_repair.py`
- **Lesson.** Writing a second tool for the same job in a different context reproduces every detail the first one learned the hard way, unless the detail is written down somewhere the second author will look. This is INC-0067 seen from the other side: there the callers were not migrated to the helper, here the helper's behaviour was not carried into the second implementation. Both are the cost of a rule living in code rather than in a statement of the rule.

INC-0071. Clauses written from a truncated report were appended to the end of the wrong word

2026-09-22, Wrong data shown or stored

- **What was seen.** Repaired choices read 'in chronological orderorder', 'return to them afterwardsards', 'how it would spend itintends to spend the money' and 'the bags the proposal would affectl would charge for'. Eleven choices across four bank files were garbled in a way that leaves the file valid JavaScript, leaves the item longer, and moves the measured distribution in the intended direction.
- **Why.** The report that shows which distractors need lengthening truncates each choice to fit a terminal line. Several clauses were written as continuations of the truncated text, on the assumption that the tool inserts at the point the needle ends. It does not: `repair()` appends at the end of the string literal, which is correct for a clause and wrong for a continuation. Nothing distinguished the two, because a continuation is just a clause whose first character is a letter.
- **How it surfaced.** Reading the source after a needle failed to match, which showed the previous pass had written 'would spend itintends to spend the money' into the same choice. (Found by reading the code or the output)
- **Fix.** `repair()` refuses a clause that does not begin with a space or punctuation, which is exactly the shape of a continuation. The four affected files were reverted and the repairs redone with clauses written to append rather than to continue.
- **What stops it now.** `bank_repair.repair` refuses a clause beginning with a letter or digit, which would run into the preceding word in `src/bank_repair.py`
- **Lesson.** A report that truncates its output invites the reader to write text that continues it, and a tool that appends will put that text somewhere else. Either the report should not truncate the field the caller has to write against, or the tool should refuse input shaped like a continuation. The cheap half is the refusal, because it is one condition and it cannot be forgotten, while remembering not to write continuations is a habit that has to hold every time.

INC-0072. A distractor was replaced and the explanation went on naming the old one

2026-09-22, Wrong data shown or stored

- **What was seen.** SR001 offers unremarkable, unaccounted for, unrelated and undisturbed, and its wrong answer note begins 'Unnoticed reverses the point'. Unnoticed is not on the paper. A student reading the explanation after answering is told why an option they were never shown is wrong.
- **Why.** The length tell on vocabulary in context items cannot be fixed by appending a clause to a single word, so `bank_repair.swap` replaces the distractor with a longer one of the same register. It replaces the text in the choices array and nowhere else. An item is not just its choices: `expl` and wrong name particular options, and on this bank they name them by word.
- **How it surfaced.** Reading the item while looking at its shape for a different purpose, three commits after the swap was made. (Found by reading the code or the output)

- **Fix.** `swap()` refuses when the text being replaced appears anywhere else inside the same item, which is exactly where an explanation that names it would be. The one affected note is rewritten to name a distractor that exists.
- **What stops it now.** `bank_repair.swap` refuses to replace a choice whose text appears elsewhere in the same item in `src/bank_repair.py`
- **Lesson.** A record has parts that refer to one another, and a tool that edits one part by text is editing a graph while looking at a string. The cheap guard is not to check every reference but to refuse the edit when the old text occurs anywhere else in the record, because that is the only place a reference to it can be. Refusing on a false positive costs one rewritten table entry; not refusing ships an explanation about an option nobody saw.

INC-0073. `check_lift` was told a one clause entry lifted two distractors

2026-09-22, Degraded

- **What was seen.** The GRE reading generator failed with three items named: GR104, GR128 and GR133 were said to have lifted 2 distractors and to belong at rank 3, and each sat at rank 4. Each of the three had exactly one clause in the table.
- **Why.** `bank_emit.extend` accepts a table entry as either one (needle, clause) pair or a list of them, and it is the only place that knows the difference. Every caller of `check_lift` then has to re-derive the count, and the obvious expression, `len(v)`, reads a bare two element tuple as two lifts. The rule for reading the table lives in `extend`; the callers each hold a private copy of it, and one copy was wrong.
- **How it surfaced.** `check_lift` failed the generator run and named all three items with their shortfalls. (A build guard caught it)
- **Fix.** `bank_emit.lift_counts(table)` derives the intent from the table using the same tuple or list test `extend` uses, and every generator that had spelled it out by hand now calls it.
- **What stops it now.** `bank_emit.lift_counts` is the single reader of the table shape, so `check_lift` and `extend` cannot disagree about what an entry says in `src/bank_emit.py`
- **Lesson.** A guard that takes the intent as an argument is only as good as the argument, and an argument derived by hand from the same data the guard is checking is a second implementation of the thing being checked. It fails in the direction that is hardest to see: too high an intent demands a rank the clauses cannot reach, and the author satisfies it by writing more clauses than the plan called for, which skews the distribution the other way while every check passes. Derive the intent from the data with the code that already reads it.

INC-0074. A corpus field written for one grammatical slot was spliced into another

2026-09-22, Wrong data shown or stored

- **What was seen.** 160 of the 833 shipped GMAT v_pc items, 19 percent, offer a choice like "That shorten the time taken to settle a claim is the most urgent of the problems facing Calloway Insurance." It is not English. A student reading it can strike it out without considering the argument, which is a free elimination on a five choice item and makes the item easier than the rating it carries.
- **Why.** A PLAN scenario stores goal as a bare infinitive phrase, because the stem it was written for reads "X intends to <goal>". Two other templates splice the same field after "has tried to", where a bare infinitive is also right. One splices it into a subject position, "That <goal> is the most urgent of the problems facing X", where English wants a gerund or a noun phrase. The field is correct; the slot it was reused in is not, and nothing connected the two.
- **How it surfaced.** Reading the output of PlanAssume while counting the PLAN corpus in order to widen it. The category had been short since it was written and the shortfall was the reason to look. (Found by reading the code or the output)
- **Fix.** Every PLAN scenario gains goal_np, the same goal as a noun phrase, and the subject slot uses it. The bare infinitive stays where a bare infinitive belongs.
- **What stops it now.** framework.verify rejects a stem or choice where That or Whether is followed directly by a bare infinitive, which is what splicing an infinitive goal into a noun slot produces in `src/gen/framework.py`
- **Lesson.** A corpus field is written against the one sentence the author had in mind, and the schema that reuses it three templates later has no way to know which shape it is. The type system says str in both places. Two things follow. Store the field in every shape a template needs and name the shapes, rather than storing one shape and trusting the next author to notice. And guard the output, not the corpus: the generated sentence is the only place the mismatch becomes visible, and a cheap pattern over the rendered text catches a class that no check on the inputs can see.

INC-0075. Every correct answer on one GMAT schema was ungrammatical, and the guard written an hour earlier could not see it

2026-09-22, Wrong data shown or stored

- **What was seen.** All 277 shipped cr_plan_eval items, a third of the GMAT Plan and Construct category, have a key reading "Whether what share of the traffic on Bridge Street stops there at all is what the measure would change." Every one of the eight scenarios produces one, across all six wh words. The student is asked to choose between four fluent distractors and one sentence that is not English, so the item is answerable without reading the argument and is rated as though it were not.
- **Why.** The same class as INC-0074 and the same corpus: check is stored as a wh clause because the explanation reads "Establish <check>, and one answer means ...". A second template put it after "Whether" and before "is what the measure would change", where a wh clause cannot go. INC-0074 was the same mistake with the goal field, found in the same reading, and the guard written for it tests for an infinitive after That or Whether. check is not an infinitive, so the guard passed it.
- **How it surfaced.** Rendering one item of each plan schema to read the output, immediately after fixing INC-0074 in the schema next to it. (Found by reading the code or the output)

- **Fix.** check is rewritten in all scenarios as a clause that reads after "Whether", the trailing "is what the measure would change" is dropped, and the explanation says "Establish whether". The guard is widened from infinitives to any corpus field spliced after That or Whether that does not read as a clause there, and it now runs over the key as well as the distractors.
- **What stops it now.** framework.check_clause_splice refuses a rendered sentence whose That or Whether is followed by a corpus field stored for another slot, tested for every plan field rather than for goal alone in `src/gen/framework.py`
- **Lesson.** A guard written from the instance in front of you covers that instance. INC-0074 was a bare infinitive in a noun slot, so the guard looked for bare infinitives, and the sentence one screen away in the same file was a wh clause in a clause slot and went straight through. The general defect was never the infinitive; it was that a corpus field carries no record of the grammatical shape it was written in, and any template may reuse it. So the guard has to be stated over the class, every field against every slot, not over the token that happened to be wrong first. The other half of this is where it was found: the distractor version was spotted first because it is louder, and the version in the key, which is three times as damaging, was found only because the first one prompted a second look. Reading one rendered item per schema would have caught both on the day they were written, and costs less than either fix.

INC-0076. str.capitalize() lowercased the rest of the sentence, and took the proper nouns with it

2026-09-22, Wrong data shown or stored

- **What was seen.** 180 shipped generated items render a proper noun in lower case. 70 of them carry it in the correct answer: "The fenwick track is the only one in the county, so every competitive runner trains there whether they win or not" and "The halloran fund supplies the starting grant for nearly all of the laboratory's work". The stem two lines above spells both correctly, so the item contradicts itself on the page and the key is the choice that looks wrong.
- **Why.** Python's str.capitalize() is not what its name suggests to a reader in a hurry. It uppercases the first character and lower cases every other one. The generators use it twenty one times to render a stored fragment as the start of a sentence, which is right for a fragment of ordinary prose and wrong for any fragment containing a name. Four corpus fields contain one.
- **How it surfaced.** Reading one rendered item from each of the five schemas in a category before widening its corpus, which is the practice INC-0075 ended with. (Found by reading the code or the output)
- **Fix.** framework.upfirst raises the first character and touches nothing else, and every call site uses it. The four affected fields need no change, which is the point: the fragment was always correct and the renderer was destroying it.
- **What stops it now.** src/build.py fails if str.capitalize() appears anywhere under src/gen, the same way it fails on an em dash in `src/build.py`

- **Lesson.** A standard library function whose name is a plausible description of half of what it does will be used for that half. `capitalize()` reads as "make this the start of a sentence" and is in fact "make this the start of a sentence and flatten everything else", and the damage is invisible until a value happens to contain a capital. The guard is not a test that the output looks right, because the output looked right for every value that had no name in it. The guard is to ban the function: the correct one is three characters of slicing, the wrong one is never what a generator wants, and a lint catches it in the diff rather than in the bank.

INC-0078. A coefficient of one was printed as 1x on 2,353 shipped items

2026-09-22, Wrong data shown or stored

- **What was seen.** 2,353 items across eight schemas and every exam that remaps them read "x squared - $1x - 30 = 0$ ", " $5(x - 7) = 1x - 6$ " and " $2x + 3y = 10$ and $1x + 5y = 12$ ". The arithmetic is right and the key is right. The notation is not how anyone writes algebra, and on a test preparation product it tells the student the question was produced by a machine that does not know the convention.
- **Why.** Every schema that draws a coefficient formats the term itself, as the number followed by the variable. That is correct for every value except one and minus one, which are written as the bare variable. The rule is a convention of notation rather than of arithmetic, so no computation check could see it, and it was reimplemented at each of the sites that needed it, each time without the exception.
- **How it surfaced.** Reading one rendered item from each of the five schemas in the last category still under target, before widening them. (Found by reading the code or the output)
- **Fix.** `framework.term` renders a coefficient and a variable together and is the only place that knows the convention. Every site that built the string itself now calls it.
- **What stops it now.** `framework` checks its own rendered stems for a unit coefficient and drops the item, so a new schema that formats a term by hand fails rather than ships in `src/gen/framework.py`
- **Lesson.** Presentation rules travel with the value, and a value formatted at the point of use is formatted by whoever was writing that line. Eight schemas each wrote the same two characters and all eight omitted the same exception, which is not eight mistakes but one missing function. The give away is the shape of the defect: identical output in unrelated files means the knowledge was never in one place. The check that catches it cannot be on the arithmetic, because the arithmetic was always right, so it has to be on the rendered string.

INC-0079. A generated schema was playable at 100 percent by picking the third shortest option, and no check looked at generated schemas

2026-09-22, Wrong data shown or stored

- **What was seen.** A student who always picks the third shortest option answers 861 of 861 sat_rw_apostrophe items correctly, and another 887 through the ACT remap, without reading a word. It is the largest single schema in Standard English Conventions on two exams. Measuring the rest found 39 of 92 generated schemas over the 1.8 times chance cap on one of the three figures, among them cr_plan_eval at 98 percent on one rank over 1,117 items, act_kol_redundancy at 100, act_s_changed at 97 percent shortest, and cr_plan_weaken at 65 percent longest.
- **Why.** INC-0069 added a length bias check per hand written source file, because a section figure hid a file at 88 percent. The same argument applies one level down and was not followed there. Generated items are checked only at the section level, and the section check runs on what the harness loads, which is the hand written banks plus the strided starter slice, so a schema contributes a few dozen items to a figure covering hundreds. The apostrophe schema is worse than an accident of drafting: its four choices are the four forms of one noun, and for a regular noun those forms are ordered by length by construction, so the key sits at the same rank on every item the schema can produce.
- **How it surfaced.** Widening five schemas failed the SAT Math ratchet, and measuring every generated schema to see whether that failure was real turned up the rest. (Found by measuring something)
- **Fix.** build_banks measures every generated schema over the whole bank rather than the starter, against a recorded table that can only come down. The apostrophe schema gains nouns whose plural is not the singular plus one character, so the four forms are no longer length ordered.
- **What stops it now.** build_banks fails on a generated schema over its recorded length bias, measured across the full bank per schema and per exam in `src/build_banks.py`
- **Lesson.** A check is scoped to a grain, and the grain is a claim about where a defect can live. INC-0069 moved the grain from the section to the file for hand written banks and stopped there, so the same defect went on living one level down in generated ones, where there are far more items. When a check finds something by being made finer, the question to ask immediately is what else is measured at the old grain. The second half is about which populations a check can see: this one ran on what the test harness loads, which is a sample chosen for a different purpose, and a sample chosen for a different purpose is not a population you can make claims about.

INC-0081. A student who always answers 1 scores 98 percent on a schema, and no check looked at the answer itself

2026-09-22, *Wrong data shown or stored*

- **What was seen.** msr_count ships 1,867 items and the correct answer is 1 on 98 percent of them: the case sets it builds almost always contain exactly one compliant row. act_s_trend ships 820 and the answer is the rising option on 76 percent, because the studies it draws trend upward three times in four. gt_count is at 47 percent on 1, and three Data Sufficiency schemas sit at 42 to 45 percent on one of their five fixed statements against a chance rate of 20.

- **Why.** Three checks look at where the answer sits: the position among the choices, the length rank, and for numeric items the value rank. All three are about the answer's place in the set it was shown in. None looks at the answer itself. A schema whose parameters happen to produce the same answer over and over passes every one of them, because the answer moves around the choices perfectly well; it is only always the same answer.
- **How it surfaced.** Dumping a dozen `msr_count` items to work out why its value rank would not move after two new distractors, and noticing that every key was 1. (Found by reading the code or the output)
- **Fix.** `build_banks` measures the most common single answer value per schema and refuses one over its recorded figure, alongside the three place based figures. `make_cases` draws how many rows comply rather than leaving it to fall out of the violation draw, and the science studies draw their direction evenly.
- **What stops it now.** `build_banks` records and ratchets the most common single answer value per schema, which is the first check on what the answer is rather than on where it sits in `src/build_banks.py`
- **Lesson.** Every guard here measured the answer's place in its set, and a set of guards that all take the same kind of measurement shares a blind spot the size of everything else. The tell they could not see was the simplest one a student would find: the answer is the same answer. When adding the third check of a kind, the question worth asking is not whether it is stricter than the other two but what all three have in common, because that is what is going unmeasured.

INC-0082. Nine published exam facts cite test prep companies, in the one published corpus with no source validator

2026-09-22, *Wrong data shown or stored*

- **What was seen.** The five exam guide pages publish nine figures sourced to test prep and content marketing companies: The Princeton Review three times (the digital SAT score release window, and the two LSAT facts that logic games are retired and that the variable section can appear anywhere), Applerouth twice (the enhanced ACT rollout dates and Science becoming optional), plus Menlo Coaching for how the GRE is delivered, Achievable for the GRE fee reduction amount, UWorld College Prep for SAT superscoring, and Sallie Mae for the SAT retake limit. CLAUDE.md bans coaching site blogs outright as sources. Several of these are structural facts a student plans around, carried under a citation the house rules forbid.
- **Why.** Two things, and the second is why the first went unseen. The published figure corpora are `data/schools`, `data/colleges` and `data/exams.json`. `validate_schools.py` and `validate_colleges.py` enforce the source policy on the first two and run from their builds; `data/exams.json` has no validator at all and `build_exams.py` calls none, so nothing has ever read a source on it. Even had one existed, it would have passed all nine: `BANNED_SOURCES` lists the six sites CLAUDE.md names as examples, and CLAUDE.md bans those six AND the category coaching site blogs. The general clause was dropped in transcription, so the list refuses GMAT Club by name and accepts Kaplan.

- **How it surfaced.** Reading ROADMAP.md for open work rather than picking by judgement. It carried a note that sat.score_release cited The Princeton Review and was still open. Asking the obvious next question, which corpus that figure lives in and what validates it, found the file had no validator; walking every url host in it against the test makers' own domains found eight more. (Found by reading the code or the output)
- **Fix.** src/validate_exams.py applies the same contract to data/exams.json that the other two corpora have had: every published figure carries src, year and url, the url host must be the test maker's own domain or an explicitly allowed one, and banned sources are fatal. build_exams.py calls it before it renders. The banned list moves to one shared module so the three validators cannot drift, and gains the coaching site category CLAUDE.md bans alongside the six named sites. Each of the nine figures was then re-verified against the test maker's own page and recited to it, or removed where the maker does not publish it.
- **What stops it now.** validate_exams.py fails the build on a figure in data/exams.json missing src, year or url, on a banned source, and on a url whose host is not the test maker's own or explicitly allowed, so a plausible looking citation from a prep company cannot be added silently in `src/validate_exams.py`
- **Lesson.** Two lessons, and they compound. A rule copied into code by its examples loses the clause the examples were illustrating: CLAUDE.md bans six named sites and coaching site blogs, and the list kept the six and dropped the category, which is the half that generalises. And a validator gets written for the corpus that had the problem at the time, then quietly defines what is checked: two of three published corpora were enforced and the third had never had a source read, which is not a weaker check but an absent one. When a guard exists, the question is not only whether it is strict enough but which of the things it could be pointed at it is not pointed at.

Tests and guards (17)

INC-0016. The performance test waited for the load event, which waits for the thing being optimised

2026-09-19, Silent loss, `4b25169` PR #45

- **What was seen.** A change that made the page usable nine times faster would have reported no improvement.
- **Why.** smoke_load waited for the load event. The load event waits for the async bank, which is precisely the part the fix moved off the critical path.
- **How it surfaced.** The number refused to move when it obviously should have. (Found by measuring something)
- **Fix.** Wait for the first question to be answerable.
- **What stops it now.** smoke_load waits on the user-visible milestone in `src/smoke_load.js`

- **Cost.** would have hidden the entire win
- **Lesson.** Measure the moment the user can act, not a browser lifecycle event. A test that measures the wrong instant is worse than no test, because it produces a number people trust.

INC-0017. A build guard silently narrowed its own scope when the counts grew

2026-09-19, Silent loss, `4b25169` PR #45

- **What was seen.** The item-count guard kept passing while checking nothing.
- **Why.** The guard matched `\d{2,4}`. When the banks passed ten thousand, the two largest counts stopped matching the pattern and fell out of the check.
- **How it surfaced.** Reading the guard while raising the counts. (Found by reading the code or the output)
- **Fix.** Match any run of digits.
- **What stops it now.** the guard asserts how many counts it checked in `src/build.py`
- **Cost.** the two largest banks unchecked
- **Lesson.** A regex with a length bound is a guard with an expiry date. Assert the number of things checked, not only that the checks passed.

INC-0019. The audit reported a working link as broken

2026-09-19, Cosmetic, `a34a798` PR #47

- **What was seen.** A false positive in the launch audit.
- **Why.** Relative link resolution in the audit did not match the browser's.
- **How it surfaced.** Checking a reported failure by hand. (Found by reading the code or the output)
- **Fix.** Resolve links the way the browser does.
- **What stops it now.** Nothing automated. This one is still carried by attention.
- **Cost.** trust in the audit
- **Lesson.** Check your checker. A tool that cries wolf gets muted, and then it is worse than nothing.

INC-0021. A test passed because nothing was configured at all

2026-09-19, Silent loss, `489363b` PR #46

- **What was seen.** `smoke_funnel`'s 'nothing sent on refusal' assertion passed. It would have passed with the feature deleted.
- **Why.** The test used storage key `sfn_consent` instead of `sfn_consent_v1`, so consent was never set up, so nothing was ever sent, so the assertion held vacuously.

- **How it surfaced.** Fixing the key, which immediately exposed a real bug underneath. (Found by reading the code or the output)
- **Fix.** Use the real key, and assert the positive case as well as the negative.
- **What stops it now.** every 'nothing happens' assertion is paired with a 'something happens' assertion in `src/smoke_funnel.js`
- **Cost.** a real bug hidden behind a green check
- **Lesson.** A negative assertion passes when the system is broken in the right way. Always pair it with the positive case, or it is testing nothing.

INC-0026. I reported a regression that was seed noise

2026-09-21, Cosmetic, `c63654a` PR #56

- **What was seen.** I reported that SAT band coverage fell from 88 to 72 percent after an engine change.
- **Why.** I compared a single seed before and after. Across seeds 3, 7 and 11 the baseline was 52, 88 and 80 against 72, 80 and 72 for the change. The spread between seeds was larger than the effect.
- **How it surfaced.** Running more seeds because the number looked too clean. (Found by measuring something)
- **Fix.** Report across seeds, never one.
- **What stops it now.** `review_bot` runs multiple seeds and reports the spread in `src/review_bot.js`
- **Cost.** a false regression report, corrected in the same session
- **Lesson.** A stochastic measurement on one seed is an anecdote. If your system has randomness, a single-run before-and-after cannot distinguish a change from the weather.

INC-0030. The theme parity test matched by name prefix and caught a font size

2026-09-21, Cosmetic, `8af7188` PR #61

- **What was seen.** The parity guard flagged `--text-xs`, which is a length, not a colour.
- **Why.** Token selection was by name prefix rather than by whether the value is a colour.
- **How it surfaced.** Reading the failure. (Found by reading the code or the output)
- **Fix.** Select tokens by inspecting the value.
- **What stops it now.** `smoke_charts` selects by value, not name in `src/smoke_charts.js`
- **Cost.** a false failure
- **Lesson.** Classify by what a thing is, not by what it is called. Naming conventions are a hint, never a type.

INC-0031. The browser suites had never actually run, because npm install had not

2026-09-21, Silent loss, `8af7188` PR #61

- **What was seen.** Browser tests appeared to be part of the suite and were not being executed.
- **Why.** playwright was listed as a devDependency and node_modules did not exist in the environment.
- **How it surfaced.** Running them by hand and getting a module-not-found. (Found by measuring something)
- **Fix.** Add npm install and playwright install to CI, and check in package.json.
- **What stops it now.** CI runs npm run test:browser in `.github/workflows/ci.yml`
- **Cost.** an unknown number of regressions unnoticed
- **Lesson.** A test that is not wired into CI is a test that does not exist. Prove a suite runs in the place it is supposed to run, not on your machine.

INC-0033. The dashboard test measured a hidden element and passed

2026-09-21, Silent loss, `8827e64` PR #62

- **What was seen.** Layout and contrast assertions passed on a view that was never displayed.
- **Why.** The admin view carries a hidden class. A display:none element still answers innerText and getComputedStyle, so every assertion held on a box that had never been laid out.
- **How it surfaced.** Screenshotting the result and seeing the dashboard was not in it. (Found by rendering it and looking)
- **Fix.** Call show('admin') and dismiss the onboarding overlay before measuring.
- **What stops it now.** smoke_business asserts the view is on screen and taller than 400px before measuring it in `src/smoke_business.js`
- **Cost.** a layout suite that tested nothing, caught the same day
- **Lesson.** Before you measure a layout, assert the thing is rendered. Hidden elements answer most DOM questions, and they answer them wrongly.

INC-0046. The error queue was 81 percent other people's blocked scripts

2026-09-21, Degraded, `c4698e0` PR #52

- **What was seen.** 1,202 of the first 1,490 rows in the error table were one third-party beacon being blocked by ad blockers.
- **Why.** The handler captured every error on the page, including ones from scripts the site does not control and cannot fix.
- **How it surfaced.** Clustering the rows and looking at the largest cluster. (Found by measuring something)

- **Fix.** Filter third-party script errors, and give the triage queue a permanent noise label.
- **What stops it now.** the triage queue mutes a cluster marked noise in `src/app_template.html`
- **Cost.** a monitoring surface that was 81 percent noise
- **Lesson.** An error feed with no filter is a feed nobody reads. Signal has to be defended, and the cheapest defence is a human label that mutes permanently, so the queue gets quieter as it learns.

INC-0053. The playbook's own citation guard failed CI on its first run

2026-09-21, Cosmetic, PR #62

- **What was seen.** CI reported all 37 cited commits as nonexistent, with 'fatal: Not a valid object name' for every one.
- **Why.** The checkout action shallow-clones by default, so no historical commit is present in the CI working copy. The guard was correct and its environment assumption was not.
- **How it surfaced.** The first CI run of the guard. (A test caught it)
- **Fix.** fetch-depth: 0 on the checkout so the history is actually there to check.
- **What stops it now.** fetch-depth: 0 in the workflow in `.github/workflows/ci.yml`
- **Cost.** one red CI run
- **Lesson.** A guard that reads the repository needs the repository. CI checkouts are shallow by default, and anything that walks history, blames a line or resolves an old hash will fail in a way that looks like the data is wrong rather than the clone.

INC-0054. The first fix asked the wrong question and muted a working check

2026-09-21, Silent loss, PR #62

- **What was seen.** After guarding on 'is this a shallow clone', the citation check stopped running locally, where it had been working perfectly and verifying all 52 citations.
- **Why.** This clone is shallow and still holds every commit the ledger cites, which is the normal case. Shallowness does not imply missing history, so the guard disabled itself in the one environment where it was useful.
- **How it surfaced.** Reading the output after the fix and noticing a check had turned into a skip. (Found by reading the code or the output)
- **Fix.** Ask whether ANY citation resolved. None at all means the history is absent; some means those particular citations are wrong.
- **What stops it now.** the check degrades only when zero citations resolve in `src/smoke_playbook.js`
- **Cost.** a working check disabled for the length of one edit

- **Lesson.** When you add a condition that skips a check, make sure it describes the failure and not something merely correlated with it. A skip is indistinguishable from a pass in the output, so the fix for a noisy check can silently delete it.

INC-0055. A new browser suite hardcoded this machine's browser directory and crashed in CI

2026-09-21, Cosmetic, PR #62

- **What was seen.** ENOENT scandir /opt/pw-browsers on the first CI run of the dashboard suite, after every other suite had passed.
- **Why.** The suite read a browser directory that exists in the development sandbox and not on a CI runner, where Playwright installs browsers in its own location. The older suites pass process.env.CHROMIUM_PATH straight through, which is undefined in CI and correctly means 'you decide'.
- **How it surfaced.** The first CI run of the suite. (A test caught it)
- **Fix.** src/chromium_path.js, one shared resolver: the environment variable, then the sandbox directory if it exists, then undefined so Playwright resolves its own.
- **What stops it now.** one shared resolver, so the logic cannot differ between suites in `src/chromium_path.js`
- **Cost.** one red CI run
- **Lesson.** A path that exists on the machine you wrote the test on is not a path. Resolve environment-specific locations through one helper that falls back to the tool's own default, and return undefined rather than an empty string, because undefined means 'you decide' and an empty string means 'launch nothing'.

INC-0057. The ledger cited commits that squash merging destroys

2026-09-21, Silent loss, PR #63

- **What was seen.** CI failed with three assertions at once: six citations unresolvable, the commit count wrong, and the HTML naming the wrong build commit.
- **Why.** One cause behind all three. Each incident cites the commit that fixed it, and this repository squash merges, so a branch commit ceases to exist the moment its pull request lands. The builder treated an unresolvable citation as fatal, so it refused to rebuild; build.py catches that failure as a warning rather than stopping the deploy, so the stale committed artefacts stayed on disk and the two freshness assertions then failed against them.
- **How it surfaced.** The first CI run after a squash merge, which is the first moment the problem could exist. (A test caught it)

- **Fix.** Cite the pull request as the durable reference and treat the commit hash as best effort. A citation that no longer resolves is reported and allowed when the record carries a PR number, and is still fatal when it does not.
- **What stops it now.** the builder and smoke_playbook both distinguish a squashed commit from a wrong one in `src/build_playbook.py`
- **Cost.** one red CI run, and a document that silently stopped rebuilding
- **Lesson.** A commit hash is not a durable citation in a repository that squashes. Pull requests, issues and tags survive history rewriting; branch commits do not. Cite the thing that outlives the merge, and make any check of the other one advisory.

INC-0061. A repeat metric that gets worse when the bank gets better

2026-09-22, Cosmetic, PR #65

- **What was seen.** Doubling the LSAT reading bank cut the repeats a student actually sees by 46 percent, from 1875 to 1006, and the bot's avoidable repeats figure went up, from 258 to 309.
- **Why.** A repeat counts as avoidable when the student has not yet exhausted that section's pool. Enlarging the pool keeps that condition true for longer in every sitting, so the window in which a repeat is classified avoidable widens with the pool. Measured directly, only 5 of 1300 repeats in a single long run were genuinely avoidable, and both pools were exhausted.
- **How it surfaced.** The number moved the wrong way after a change that should only have helped, so the metric was read rather than trusted. (Found by measuring something)
- **Fix.** Report the repeat rate per item served alongside the avoidable count, so the student-facing number is visible next to the diagnostic one. The avoidable definition is left alone rather than quietly redefined to make a number look better.
- **What stops it now.** the bot prints repeats per item served next to the avoidable count in `src/review_bot.js`
- **Cost.** nearly a false regression, and a standing hazard for whoever reads it next
- **Lesson.** A metric that moves against you when the product improves will eventually be used to justify reverting an improvement. When a number goes the wrong way after a change that should only have helped, measure the underlying thing directly before believing either the number or your own explanation of it. Never redefine the metric in the same change that made it look bad.

INC-0065. The analysis chapter called one file eight failing guards

2026-09-22, Wrong data shown or stored, PR #68

- **What was seen.** The generated chapter reported `src/build.py` as a guard named by eight incidents, under a heading saying a guard named twice is one that did not hold. `build.py` contains dozens of unrelated guards and most of those eight are different ones.

- **Why.** The recurrence grouping keys on `guard_file`, which is the file a guard lives in, not the guard. A file that is the natural home for many checks therefore looks like a single check failing repeatedly, and the chapter states that reading in the heading as though it were established.
- **How it surfaced.** Reading the generated chapter after a build, to see whether it had picked up a genuine recurrence recorded minutes earlier. (Found by reading the code or the output)
- **Fix.** Group by the guard description, which names the mechanism, and report file level clustering separately with wording that says what it actually means.
- **What stops it now.** the analysis chapter groups recurrence by guard rather than by file in `src/build_playbook.py`
- **Cost.** a generated chapter making a confident and wrong claim about where to spend effort
- **Lesson.** An aggregate is a claim about whatever you grouped by. Group by the file and you have measured the file. State the grouping in the sentence that reports the result, and the overclaim becomes visible while you are writing it.

INC-0067. The browser path fix covered two suites and three others kept crashing

2026-09-22, Silent loss

- **What was seen.** `smoke_items.js`, `smoke_consent.js` and `smoke_billing.js` all died at launch with 'Executable doesn't exist at `/opt/pw-browsers/chromium_headless_shell-1243/...`'. The sandbox holds `chromium-1194` and `chromium_headless_shell-1194`, so Playwright's own resolved path pointed at a build that is not installed. CI passed throughout, because a CI runner installs the browser Playwright expects.
- **Why.** INC-0055 was the same failure in `smoke_business.js`, and the fix was `src/chromium_path.js`, a shared resolver that prefers `CHROMIUM_PATH`, falls back to the newest `/opt/pw-browsers/chromium-*`, and otherwise returns undefined so Playwright decides. Two suites were changed to use it, `smoke_business` and `smoke_fit`. The three older suites were left reading `process.env.CHROMIUM_PATH` directly, which with the variable unset passes `executablePath`: undefined and hands the decision back to Playwright, which is exactly the case the resolver exists to override. A shared module only helps the callers that call it.
- **How it surfaced.** Running the three suites after a bank change, as the pre push checklist requires. They had not been run in this sandbox since the resolver was introduced. (A test caught it)
- **Fix.** All ten browser suites and `playbook_pdf.js` now call `chromiumPath()` from `src/chromium_path.js`. A check in `src/build.py` fails the build if any file that launches Playwright does not, and on its first run it named six suites beyond the three the failure had surfaced, plus four more that a grep written from its output then found.
- **What stops it now.** the build refuses a Playwright launch that reads `CHROMIUM_PATH` instead of calling `chromiumPath()` in `src/build.py`

- **Lesson.** Extracting a shared helper does not migrate the callers. The extraction fixes the file it was extracted from and leaves every sibling on the old path, which is INC-0059 and INC-0064 in a different costume: a correction applied to the instances in hand rather than to the pattern. Three suites had failed visibly and ten were wrong; the seven silent ones were found by the guard, not by reading. When a helper exists because a direct call was wrong, make the direct call fail the build, and let it enumerate the callers rather than enumerating them by hand.

INC-0077. A review bot check whose verdict was three coin flips warned on an unrelated bank change

2026-09-22, *Cosmetic*

- **What was seen.** Widening two GMAT critical reasoning corpora turned the bot's planted weakness check from ok to warn: 2 of 3 sections correctly identified as weakest. Nothing about the diagnosis had changed. The bot threads one seeded generator through every check in an exam, so more items meant the earlier checks consumed a different number of draws and the planted weakness test started from a different point in the stream.
- **Why.** The check sat one simulated student per section, three sittings in total, and reported a pass only if all three came out right. Measured over 60 independent seeds the diagnosis is right on 179 of 180 sittings, so at three sittings the check warns on roughly three percent of seeds by sampling alone. Every other check in the bot already averages over 25 sittings; this one did not, and its verdict was therefore a property of the seed as much as of the engine.
- **How it surfaced.** The warning appeared on a change that could not plausibly have caused it, so the check was measured across seeds instead of being believed or dismissed. (Found by measuring something)
- **Fix.** The check sits five students per section and reports the rate, passing at 90 percent or better and failing below half. The threshold is stated against the measured 99 percent so the margin is visible rather than implied.
- **What stops it now.** the planted weakness check reports a rate over 15 sittings rather than a verdict over 3 in `src/review_bot.js`
- **Lesson.** A check that reports pass or fail from a handful of random draws is a check that will flip on work that has nothing to do with it, and the cost is not the false alarm. It is that the next real alarm arrives in a tool people have learned to shrug at. Before believing or dismissing a warning, run the thing it measures enough times to know its rate: that answers both whether this alarm is real and whether the check is worth keeping in its current form. Here the answer was that the engine was fine and the check was wrong, and both were worth knowing.

Front end (8)

INC-0001. Unescaped quotes in onclick strings took the whole app down

2026-08-19, Site down, `1f3ecec` PR #7

- **What was seen.** On the deployed site no tab rendered and every nav click was dead. The page looked fine; nothing worked.
- **Why.** Five onclick handlers were built as JS strings containing unescaped single quotes inside single-quoted string literals. That is a parse error, and a parse error anywhere in an inline script kills the entire script, not the one handler.
- **How it surfaced.** Reported from the live site after deploy. (A person hit it)
- **Fix.** Escape the quotes, and make the build parse every inline script with node before it will write a page.
- **What stops it now.** build.py node-parses every inline script and fails the build on a syntax error in `src/build.py`
- **Cost.** a live outage of the trainer
- **Lesson.** Generated code is code. If your build writes JavaScript into a string, the build must parse the result, because the blast radius of one bad character is the whole file, not the line.

INC-0038. One exam's progress blob overwrote another's

2026-09-16, Wrong data shown or stored, `81eecec` PR #30

- **What was seen.** None shipped: caught before release. A student practising two exams would have had one exam's progress destroyed by the other.
- **Why.** Progress was stored under a single key. Making the engine multi-exam added a second writer to the same slot without keying the slot by exam.
- **How it surfaced.** Reviewing the cross-exam data paths while adding the second exam. (Found by reading the code or the output)
- **Fix.** Key the stored state per exam.
- **What stops it now.** engine tests run once per exam and assert progress isolation in `src/test.js`
- **Cost.** would have destroyed real progress
- **Lesson.** The moment a single-tenant store becomes multi-tenant, every key in it is a collision waiting to happen. Enumerate the writers before you add the second tenant, not after.

INC-0043. The GRE app told GRE students to calibrate at the wrong test maker's site

2026-09-16, Wrong data shown or stored, `1a003bb` PR #34

- **What was seen.** A shipped application displayed a score card headed with a different exam's name and linked students to that exam's official site.
- **Why.** The template forked on the exam id in seven places and treated not-SAT as the original exam. Adding a third exam made every one of those branches wrong.
- **How it surfaced.** Reading the built page for the third exam. (Found by reading the code or the output)
- **Fix.** Per-exam copy moved into the registry and resolved from the injected exam id.
- **What stops it now.** a headless check asserts each app names itself in `src/test.js`
- **Cost.** a live app misidentifying itself
- **Lesson.** A conditional that treats not-A as the original case is a bug the day a third case exists. Resolve variants from data, and the third one costs a row rather than a search.

INC-0015. The whole bank blocked first paint: 20 seconds to the first question on 3G

2026-09-19, Degraded, `4b25169` PR #45

- **What was seen.** On throttled regular 3G, 20.3 seconds to the first GMAT question and 23.4 for ACT.
- **Why.** The entire item bank shipped as a blocking script. It grew past ten megabytes without anyone measuring what that cost.
- **How it surfaced.** A throttled measurement, added because the number had never been taken. (Found by measuring something)
- **Fix.** A strided starter bank plus an async remainder that pushes into the same array. ACT 7.6s, GMAT 11.0s; blocking download 11.3MB to 1.2MB.
- **What stops it now.** `smoke_load` measures time to first question under throttling in `src/smoke_load.js`
- **Cost.** every visitor on a slow connection, for weeks
- **Lesson.** Nobody notices a page getting slower one commit at a time. Put the number in a test the first time you care about it, not the first time somebody complains.

INC-0022. A try/catch caught nothing because the call inside it does not throw

2026-09-21, Silent loss, `c4698e0` PR #52

- **What was seen.** An error path never ran.
- **Why.** The wrapped call signals failure by return value, not by throwing, so the catch was unreachable.
- **How it surfaced.** A review bot playing a full exam against the real engine. (Found by a review bot or an adversarial pass)
- **Fix.** Check the return value.

- **What stops it now.** `review_bot.js` sits each exam repeatedly against the real engine in `src/review_bot.js`
- **Cost.** a silent failure path
- **Lesson.** `try/catch` around an API that returns errors is decoration. Know which convention each call uses before you wrap it.

INC-0028. The service worker precache regex did not match the chunked files

2026-09-21, Degraded, `58f3a93` PR #59

- **What was seen.** Chunked bank files would not have been precached offline.
- **Why.** The IMMUTABLE pattern was `/bank_rest\.js$/`, which does not match `bank_rest1.js`.
- **How it surfaced.** Reading the service worker while changing the file naming. (Found by reading the code or the output)
- **Fix.** `/bank_rest\d*\.js$/`.
- **What stops it now.** `smoke_offline` asserts every bank chunk is precached in `src/smoke_offline.js`
- **Cost.** offline would have silently degraded
- **Lesson.** When you change a filename scheme, grep for the old name as a string, including inside regexes. A pattern is a hardcoded name wearing a disguise.

INC-0032. ARR was rounded to whole dollars and lost real money at small scale

2026-09-21, Wrong data shown or stored, `8827e64` PR #62

- **What was seen.** \$159.84 of ARR printed as \$160.
- **Why.** The currency formatter was called with zero decimal places for the larger tiles, a rule chosen for readability at scale and applied at a scale where cents still matter.
- **How it surfaced.** A smoke test whose expected values were computed by hand before the page was written. (A test caught it)
- **Fix.** Keep cents below a thousand dollars, drop them above.
- **What stops it now.** `smoke_business` asserts the exact printed figures in `src/smoke_business.js`
- **Cost.** a wrong figure on the tile the owner reads most closely
- **Lesson.** A fixed rounding rule is wrong at one end of the range or the other. Pick the precision from the magnitude, and write the expected number down before you write the code that produces it.

INC-0045. The error reporter reported its own failures, in a loop

2026-09-21, Degraded, `c4698e0` PR #52

- **What was seen.** 267 rows in the error table, one per page a crawler swept.
- **Why.** fetch rejects on a network error, it does not throw, so the try/catch around the reporting call caught nothing. The rejected promise reached the unhandledrejection handler, which called the reporter again. The analytics beacon had the identical trap.
- **How it surfaced.** Reading the error table and noticing the shape of the rows. (Found by measuring something)
- **Fix.** Handle the rejection, and never report from inside the reporter.
- **What stops it now.** Nothing automated. This one is still carried by attention.
- **Cost.** 267 junk rows and a self-sustaining loop
- **Lesson.** Anything that reports failures must not be able to report its own. Check whether each call rejects or throws before you wrap it, and make the reporting path unable to re-enter itself.

Build system (6)

INC-0059. The item counter missed a whole bank file because it assumed a quoting style

2026-09-22, Silent loss, PR #65

- **What was seen.** A new bank added 35 items and the build reported the exam's total unchanged at 65. The engine tests, which load the bank for real, saw all 100.
- **Why.** The count is a regex over the concatenated source looking for an id in single quotes. The new file emits JSON escaped strings, so its ids are double quoted and none of them matched. Nothing compared the regex count to the number of items that actually load.
- **How it surfaced.** Noticing that the build summary and the test output disagreed about the same bank. (Found by measuring something)
- **Fix.** Accept either quoting style, and assert that every bank file listed for an exam contributes at least one counted item, so a file the pattern cannot see fails the build instead of counting zero.
- **What stops it now.** per-file contribution assertion in the bank counter in `src/build.py`
- **Cost.** a published item count 35 short, and a guard that would have kept getting quieter
- **Lesson.** A regex that counts things assumes a formatting convention, and a file that legitimately breaks the convention counts as zero rather than as an error. Any counter that can return zero for a non-empty input needs a per-source assertion, not just a total.

INC-0060. Nothing parsed the one file every user downloads

2026-09-22, Site down, PR #65

- **What was seen.** None shipped. A bank file with a syntax error built cleanly and the build exited zero, leaving invalid JavaScript in the bank the trainer loads on every visit.
- **Why.** The build parses every inline script in every built page, a guard added after an unescaped quote took the whole trainer down at parse time. The item bank is not an inline script. It ships as a separate file and was never in the list, so the largest generated artefact on the site was the one thing the parser never saw.
- **How it surfaced.** Deliberately breaking a bank file to test an unrelated guard, and noticing the build passed. (Found by a review bot or an adversarial pass)
- **Fix.** Parse every built bank file and every chunk of it, alongside the pages.
- **What stops it now.** the build node-parses each built bank and chunk in `src/build.py`
- **Cost.** would have broken every trainer on the next bank edit
- **Lesson.** A parse guard covers the file shapes someone thought of. When the same code moves into a new shape, a separate file, a chunk, a worker, the guard does not follow it. List what the guard covers against what the deploy actually ships, and check the difference rather than the intention.

INC-0063. The stale count guard checked two nouns and the page used a third

2026-09-22, Wrong data shown or stored, PR #67

- **What was seen.** `llms.txt` told every model that reads it the LSAT bank holds 65 questions. It held 142. The build's count guard passed the file.
- **Why.** The guard matches a number followed by original or flashcards. The LSAT line was phrased as 65 questions today, deliberately, because the number was small and the page said so plainly. The phrasing that made the sentence honest is what put it outside the pattern.
- **How it surfaced.** Reading the file while fixing a different count the guard did catch, then watching the widened guard flag a correct sourced figure. (Found by reading the code or the output)
- **Fix.** Match a second, unambiguous pattern rather than more nouns. Widening the noun list to items and questions did catch the stale figure, and immediately raised a false positive on '64 questions', which is the real GMAT Focus question count from GMAC and not a bank size at all. A checker that cries wolf gets muted, so the guard now matches 'N original', 'N flashcards' and the specific phrase 'item bank is N', which names our bank and cannot match an exam fact.
- **What stops it now.** the count guard matches original, flashcards, items and questions in `src/build.py`
- **Cost.** a published figure less than half the true one, on the page written for machines
- **Lesson.** A guard keyed to wording is a guard on the wording, not the fact, and every synonym is a hole in it. Widening the wording is the obvious repair and it trades missed defects for false alarms, which cost more because they get the guard switched off. Match a phrase that only the thing you care about can produce, rather than every word it might happen to use.

INC-0064. The guard against a blind counter was itself blind to three exams

2026-09-22, Silent loss, PR #68

- **What was seen.** A new GRE reading bank of 26 items was registered, built, and counted as zero. The published GRE total stayed at 19,888. The guard written a day earlier to catch exactly this said nothing.
- **Why.** Two failures of the same shape, one inside the other. The GRE id pattern was G[QVE] and the new ids begin GR, so the counter could not see them. And the per-file guard added for INC-0059, which asserts that every listed bank file contributes at least one counted item, looped over only the two exams that happened to be in hand when it was written.
- **How it surfaced.** Predicting it from the id pattern before building, then watching the build confirm it by reporting the old total. (Found by reading the code or the output)
- **Fix.** Add R to the GRE pattern, and loop the guard over all five exams rather than two.
- **What stops it now.** the per-file counter guard covers every exam in APPS in `src/build.py`
- **Cost.** 26 items invisible to every published count, caught before merge
- **Lesson.** A guard that covers a subset of cases reproduces the original defect in the cases it skips, and it is more dangerous than no guard because the incident it was written for feels closed. When you add a check, enumerate everything of that kind and cover all of it, or state in the code which cases are deliberately excluded and why.

INC-0080. A build step that fails while the build exits zero, and a verification run that was a remembered subset

2026-09-22, Degraded

- **What was seen.** Three commits shipped with a playbook that had not been rebuilt, because the ledger no longer checked out: INC-0077 carried an area of tooling, which is not one of the twelve the builder accepts. The site build printed WARNING and exited zero, so nothing local said no. CI said no, on the commit after the one that broke it.
- **Why.** Two things had to be true at once. The playbook build is deliberately non fatal, because playbook/ is excluded from the deploy and a broken chapter should not stop the site shipping, and it says so with a line beginning WARNING. And the run that was supposed to catch it was four smoke suites chosen from memory rather than the eight steps CI actually runs, so smoke_playbook, which does check this, was never run. Either alone is survivable. Together they mean a failure that is reported, in a form nobody was reading, to a process that had already decided what to look at.
- **How it surfaced.** The build-and-test job failed on GitHub with five playbook assertions, three commits after the one that introduced the bad row. (A test caught it)
- **Fix.** The area is corrected to testing, which is the label the builder has for a check. The non fatal line now begins ERROR rather than WARNING, because it is one: the exit code stays zero so the site still builds, and the word matches what happened so a reader scanning for failures finds it.

- **What stops it now.** the playbook build failure is reported as ERROR while still not blocking the site build, so a scan for failures catches it in `src/build.py`
- **Lesson.** Two habits, both mine rather than the code's. Verify with the sequence the pipeline runs, read out of its config, not with the subset you remember: a suite chosen from memory drifts to the parts that were failing last week. And when a step is deliberately non fatal, the word it fails with is the whole of its signal, so it has to be the word people grep for. WARNING on a line that means a deliverable did not build is an invitation to miss it, and the cost of saying ERROR while still exiting zero is nothing at all.

INC-0083. The rules digest promises to be prompt sized and its generator grows without bound

2026-09-22, Degraded

- **What was seen.** smoke_playbook failed with 'the digest stays prompt sized (4046 words)' against a 4000 word limit, on a commit whose only relation to the playbook was adding one incident. The bootstrap pack's RULES_DIGEST.md is the file a new project pastes into its prompt, so crossing the limit is the one failure that makes the deliverable useless for its purpose.
- **Why.** rules_digest prints every incident's lesson field in full, so the digest grows linearly with the ledger while the guard holds a fixed bound. At 82 incidents the lessons alone were 3,742 words against a 4,000 word budget. Nothing about the triggering commit was special: the next incident would have tripped it whoever wrote it, which is the signature of a bound asserted on an output nobody bounded.
- **How it surfaced.** test (A test caught it)
- **Fix.** The digest now prints the operative rule rather than the whole lesson: sentences from the front of the lesson until it reads as a complete thought, at least 80 characters. That is 2,119 words instead of 3,742, and 37 of the 82 lessons are already one sentence and are unchanged. The 80 character floor is not cosmetic: the companion guard asserts every lesson reaches the digest by looking for its first 60 characters, and 27 lessons have a first sentence shorter than that, so truncating at the first full stop would have satisfied one guard by breaking the other. The reasoning behind each rule stays in BUILD_PLAYBOOK.md, which the digest already tells the reader to consult.
- **What stops it now.** smoke_playbook already held the bound; what it lacked was a generator that respects one. The word count is now roughly flat per incident rather than proportional to the lesson someone happened to write in `src/build_playbook.py`
- **Lesson.** A size limit on a generated file is only a guard if something bounds the generator too; otherwise it is a delayed failure that lands on whoever commits next, and reads as their fault. When two guards constrain the same output, check the fix against both: shortening this file to satisfy the size check would have broken the completeness check that reads its first 60 characters.

CSS and layout (5)

INC-0050. A landing-page icon referenced a colour token that did not exist

2026-08-19, Cosmetic, 5337dfd

- **What was seen.** One game icon on the landing grid rendered without its colour.
- **Why.** An undefined custom property, which CSS treats as a fallback rather than an error.
- **How it surfaced.** Looking at the page on a phone during a mobile pass. (Found by rendering it and looking)
- **Fix.** Use a defined token.
- **What stops it now.** weekly_audit checks computed colour on every page in `src/weekly_audit.js`
- **Cost.** one icon, and the same root cause later cost a compliance control
- **Lesson.** The same undefined-property failure will find you repeatedly, at every severity from one icon to an invisible legal control. One audit of computed colour catches the whole class.

INC-0010. CSS comments do not nest, and one placeholder killed the palette on 1451 pages

2026-09-17, Degraded, 61c3ed2 PR #40

- **What was seen.** The colour palette was gone from every college page. It looked like a design problem and was chased as one for hours.
- **Why.** A `{{CHROME_CSS}}` placeholder was left inside an opening CSS comment in `rankings_base.css`. The first `*/` closes the outer comment, so everything after it was live CSS that overwrote the palette.
- **How it surfaced.** Eventually, by reading the built CSS rather than the template. (Found by reading the code or the output)
- **Fix.** Remove the nested comment.
- **What stops it now.** Nothing automated. This one is still carried by attention.
- **Cost.** hours, on 1451 pages
- **Lesson.** CSS comments do not nest. When a whole page category loses its styling, read the built artefact, not the source that produced it.

INC-0048. Error recovery in the CSS parser swallowed the palette silently

2026-09-17, Degraded, a920f5c PR #41

- **What was seen.** Score bars rendered as empty gaps and panels lost their borders on 1,451 college pages and the rankings.

- **Why.** A nested comment ended early, the leftover words became a selector, and the CSS parser's error recovery discarded the `:root` colour block that followed it. Every grey and navy variable on those pages resolved to nothing. There is no error anywhere in this chain; CSS is specified to recover and continue.
- **How it surfaced.** The missing borders eventually gave it away, after the missing colours did not. (Found by reading the code or the output)
- **Fix.** Remove the nested comment.
- **What stops it now.** `weekly_audit` checks computed colour on every page in `src/weekly_audit.js`
- **Cost.** the palette on over 1,450 pages
- **Lesson.** CSS never fails loudly. A malformed rule is skipped, a bad selector eats the block after it, and an undefined variable paints as nothing. Anything that matters visually has to be asserted on the rendered page, because the parser will not tell you.

INC-0018. The Do Not Sell button was invisible from the day it shipped

2026-09-19, Degraded, `a34a798` PR #47

- **What was seen.** White text on a transparent background on a white page. A legally required control that nobody could see.
- **Why.** The page referenced nine colour custom properties that exist nowhere. CSS does not treat an undefined custom property as an error; it falls back to the initial value and paints.
- **How it surfaced.** An automated contrast audit across every page. (A test caught it)
- **Fix.** Define the tokens and fix seven contrast failures, all site wide because all lived in shared chrome.
- **What stops it now.** `weekly_audit` checks computed contrast on every page in `src/weekly_audit.js`
- **Cost.** an invisible compliance control for the life of the page
- **Lesson.** An undefined CSS custom property is silent. Audit computed colour, not authored colour, and do it on the rendered page.

INC-0029. A sequential colour ramp inverts direction between themes, and the label text did not

2026-09-21, Degraded, `8af7188` PR #61

- **What was seen.** White heatmap labels on a light cell in dark mode.
- **Why.** A sequential ramp runs light to dark in a light theme and dark to light in a dark theme. The label colour was hardcoded white, so it was correct in one theme and unreadable in the other.
- **How it surfaced.** Rendering the chart in dark mode and looking at it. (Found by rendering it and looking)

- **Fix.** An `--sfnc-on-seq` token that flips with the theme.
- **What stops it now.** `smoke_charts` asserts token parity between the light and dark blocks in `src/smoke_charts.js`
- **Cost.** an unreadable chart in one theme
- **Lesson.** A dark mode does not break by having a wrong colour. It breaks by missing one. Any colour paired with a ramp has to move with the ramp.

Payments (5)

INC-0041. A signed-in user could grant themselves a paid plan

2026-09-16, Wrong data shown or stored, `81eecec` PR #30

- **What was seen.** The plan column was writable by the authenticated role, so any signed-in browser session could set its own plan to the top tier.
- **Why.** Column-level grants on the profiles table included the billing columns. Row Level Security controls which rows you can touch, not which columns.
- **How it surfaced.** Auditing the grants rather than the policies. (Found by reading the code or the output)
- **Fix.** Remove the billing columns from the grants entirely. Only the service role, used by the webhook, can write them.
- **What stops it now.** the billing columns are absent from the column grants in `supabase/migrations/20260916_harden_plan_column_and_rpc_surface.sql`
- **Cost.** free access to every paid tier
- **Lesson.** Row Level Security is row-level. Which columns a role may write is a separate grant, and anything money depends on belongs to the service role alone.

INC-0042. Every trialing subscriber would have been left on the free plan

2026-09-16, Wrong data shown or stored, `81eecec` PR #30

- **What was seen.** A checkout that starts a free trial did not upgrade the account.
- **Why.** The handler checked for a payment status of paid. A session that starts a trial settles as no payment required, which is not paid.
- **How it surfaced.** Reading the processor's status vocabulary rather than assuming it. (Found by reading the code or the output)
- **Fix.** Accept both settled states.

- **What stops it now.** `smoke_billing` covers the trial path in both auth states in `src/smoke_billing.js`
- **Cost.** every trial would have failed to grant access
- **Lesson.** Enumerate every value a third-party status field can take before you branch on one of them. The value you did not think of is usually the one that matters commercially.

INC-0035. Two Stripe event types announce one new subscription

2026-09-21, Wrong data shown or stored, 8827e64 PR #62

- **What was seen.** None: caught in design. Logging new subscriptions from both `checkout.session.completed` and `customer.subscription.created` would have doubled new MRR.
- **Why.** Stripe announces a new subscription through both events, with different event ids, in no guaranteed order.
- **How it surfaced.** Working out the idempotency key before writing the handler. (Found by reading the code or the output)
- **Fix.** A partial unique index on (source, subscription_id, kind) for the once-per-lifetime kinds, so the database decides rather than whichever event arrived first.
- **What stops it now.** `billing_events_once_idx` in `supabase/migrations/20260921_billing_events_ledger.sql`
- **Cost.** would have doubled the headline revenue figure
- **Lesson.** Idempotency belongs in the database, not in the handler. Handlers race; unique indexes do not.

INC-0036. `charge.amount_refunded` is cumulative, so partial refunds double-count

2026-09-21, Wrong data shown or stored, 8827e64 PR #62

- **What was seen.** None: caught in design. A second partial refund reports the running total, which would have counted the first refund twice.
- **Why.** Stripe's charge object reports total refunded to date, not the amount of this refund.
- **How it surfaced.** Reading the field's semantics rather than its name. (Found by reading the code or the output)
- **Fix.** Use the most recent entry in the refunds list, falling back to the cumulative total for the ordinary single full refund.
- **What stops it now.** Nothing automated. This one is still carried by attention.
- **Cost.** would have overstated refunds
- **Lesson.** Read what a payment field means, not what it is called. Cumulative and incremental fields look identical until the second event.

INC-0037. Cancellation erases the number you need to record the cancellation

2026-09-21, Wrong data shown or stored, `8827e64` PR #62

- **What was seen.** None: caught in design. Churned MRR would always have been zero.
- **Why.** The profile write on cancellation nulls `plan_amount_cents` and `plan_interval`. That row was the only place the departing subscriber's price still existed.
- **How it surfaced.** Tracing the order of writes in the handler. (Found by reading the code or the output)
- **Fix.** Read the prior profile state before the cancellation write, and log what they were paying.
- **What stops it now.** `priorState()` is called before `writeProfile` in both the deleted and updated branches in `supabase/functions/stripe-webhook/index.ts`
- **Cost.** would have reported zero churn forever
- **Lesson.** When a write is destructive, capture what you need from the old value first. Ask what question you will want to answer after this row is gone.

Infrastructure and deploy (5)

INC-0023. www and the apex were two origins, so consent and rankings split in half

2026-09-21, Wrong data shown or stored, `b852727` PR #54

- **What was seen.** Two hostnames served the same site. `localStorage` is per origin, so a visitor who arrived on both had two consent states and their ranking signal was split.
- **Why.** No canonical host redirect. The Cloudflare connector available here has no zone-level tools, so it had to be solved in the Worker.
- **How it surfaced.** Reasoning about origin scoping while investigating traffic. (Found by reading the code or the output)
- **Fix.** `src/worker.mjs` 301s `www` to the apex, with `run_worker_first` true so the asset router does not answer first.
- **What stops it now.** `smoke_redirect` asserts the redirect and that `run_worker_first` stays true in `src/smoke_redirect.js`
- **Cost.** split consent state and diluted ranking signal
- **Lesson.** Two hostnames are two origins and therefore two of everything the browser scopes by origin. Pick one and redirect on the server, not in a meta tag.

INC-0024. The .git directory was being served on production

2026-09-21, Wrong data shown or stored, `07f0646` PR #55

- **What was seen.** <https://startfromnowhere.com/.git/config> returned 200.
- **Why.** `.assetsignore` excluded `src/`, `data/` and `supabase/` but not `**/.git`. Cloudflare's asset uploader walks the repository root, which is the `assets` directory.
- **How it surfaced.** An adversarial review of the deploy exclusion list, after the first version of that list had already shipped. (Found by a review bot or an adversarial pass)
- **Fix.** Exclude `**/.git`, and rewrite the guard to derive what must be served from real references rather than a hand-written allowlist.
- **What stops it now.** `smoke_redirect` derives the must-serve set with `git check-ignore` and asserts the source is excluded in `src/smoke_redirect.js`
- **Cost.** the full repository history publicly readable
- **Lesson.** An exclusion list is a denylist, and denylists are wrong by omission. Derive the allowed set from what the built pages actually reference.

INC-0025. A stale edge cache made a fixed 404 look like a live 200

2026-09-21, *Cosmetic*, `07f0646` PR #55

- **What was seen.** After the fix deployed, `/.git/config` still returned 200.
- **Why.** Cloudflare had cached the response with `max-age=300`. The fix was live; the verification was reading the cache.
- **How it surfaced.** Re-requesting with a cache-busting query string, which returned 404. (Found by measuring something)
- **Fix.** None needed in code.
- **What stops it now.** Nothing automated. This one is still carried by attention.
- **Cost.** minutes, and nearly a wrong conclusion
- **Lesson.** Verify a cache-fronted fix with a cache-busting request, or you are testing the cache. This costs one query parameter and saves an hour of chasing a fix that already worked.

INC-0027. A 29.88 MiB bank file would have failed the deploy the moment it merged

2026-09-21, *Site down*, `58f3a93` PR #59

- **What was seen.** None yet. The build succeeded locally and the deploy would have rejected the file.
- **Why.** Cloudflare Workers static assets cap a single file at 25 MiB. Raising the generated banks pushed the ACT rest-bank past it.
- **How it surfaced.** Checking the platform limit against the real output size before pushing. (Found by reading the code or the output)
- **Fix.** Chunk the rest-bank at 18 MiB per file and glob the chunks in numeric order.
- **What stops it now.** the build fails on any asset over 25 MiB in `src/build.py`

- **Cost.** would have broken the deploy
- **Lesson.** Know your platform's hard limits and assert them in the build. A deploy-time rejection is a bad place to learn a number your build could have told you.

INC-0047. Excluding the design directory would have shipped the legal pages with no styling

2026-09-21, Degraded, 07f0646 PR #55

- **What was seen.** None: caught in draft. Terms and Privacy would have rendered with no font, no colour and no background.
- **Why.** Those two pages load a stylesheet from the design directory that is nothing but imports of the token files. Excluding the directory from the deploy removed both the stylesheet and the tokens it imports.
- **How it surfaced.** An adversarial re-read of the exclusion list against what the built pages actually reference. (Found by a review bot or an adversarial pass)
- **Fix.** Re-include the tokens and the stylesheet, and rewrite the guard to read every href and src out of the built pages and follow one level of CSS import, instead of comparing against a list kept by hand.
- **What stops it now.** smoke_redirect derives the must-serve set from the built pages in `src/smoke_redirect.js`
- **Cost.** two legal pages unstyled, caught before deploy
- **Lesson.** A guard that is a hand-kept list of safe paths is written by the same hand that made the mistake. Derive the safe set from the artefact, not from memory.

Scoring and selection (4)

INC-0040. The easier module was not easier

2026-09-16, Wrong data shown or stored, 81eecec PR #30

- **What was seen.** An adaptive test routed weaker students to a second module that was supposed to be easier, and was not.
- **Why.** The two modules were assembled without comparing their difficulty distributions.
- **How it surfaced.** Measuring the mean difficulty of each module. (Found by measuring something)
- **Fix.** Assemble the modules against a difficulty target and assert it.
- **What stops it now.** module difficulty is asserted at build time in `src/test.js`
- **Cost.** adaptive routing that did nothing

- **Lesson.** If your system branches on difficulty, measure that the branches actually differ. A label is not a property.

INC-0005. The adaptive engine was learning from nobody

2026-09-17, Silent loss, `fc4114c` PR #39

- **What was seen.** The attempts table held zero rows in production while the app was plainly being used.
- **Why.** `Cloud.logAttempt` returns early without a signed-in user, and the product deliberately requires no account. So the one path that recorded answers was the one path most users never took.
- **How it surfaced.** Querying the production table and finding it empty. (Found by measuring something)
- **Fix.** `item_events`: an unlinkable telemetry table with no user, session, device or address column, which is exactly why it needs no consent gate.
- **What stops it now.** `smoke_consent` asserts the telemetry body carries no identifying field in `src/smoke_consent.js`
- **Cost.** weeks of telemetry never recorded
- **Lesson.** Check that your instrumentation fired at all before you trust anything built on it. An empty table looks identical to a quiet week.

INC-0006. Sentence equivalence items were unanswerable however well you answered

2026-09-17, Wrong data shown or stored, `fc4114c` PR #39

- **What was seen.** Every GRE sentence equivalence item graded as wrong.
- **Why.** The items carry a two-element key and the engine graded by requiring an array of two, but `answerInputs` had no branch for that type, so they rendered as single pick and returned a scalar.
- **How it surfaced.** Rendering and grading one item of every schema. (A test caught it)
- **Fix.** Add the missing input branch.
- **What stops it now.** `smoke_items` renders and grades one item from all 182 generator schemas at phone width in `src/smoke_items.js`
- **Cost.** eleven items unusable since they shipped
- **Lesson.** A type system spread across a renderer and a grader will drift. The cheapest guard is one that exercises every variant end to end, once.

INC-0051. Fixing repeats directly left a side door through passage groups

2026-09-21, Degraded, `65bd09d` PR #58

- **What was seen.** After a freshness gate cut avoidable repeats from 276 to 9 on one exam, one exam still served 117 and another 941 across 25 sittings.
- **Why.** The gate picked an unserved anchor item, and then the group expansion pulled in every question sharing that passage, including the ones already answered. The gate was applied to the anchor and not to what the anchor dragged with it.
- **How it surfaced.** A review bot counting avoidable repeats across 25 sittings per exam. (Found by a review bot or an adversarial pass)
- **Fix.** The group keeps the anchor and any unseen sibling and drops the rest, so a group can never come back empty.
- **What stops it now.** the review bot counts avoidable repeats and the number is pinned in `src/review_bot.js`
- **Cost.** repeats persisting on two exams after the fix was believed complete
- **Lesson.** When you add a filter, find every path that adds items after the filter runs. A gate on the entry point is not a gate on the set.

Database (4)

INC-0012. Postgres fired triggers alphabetically and revoked every adult opt-in

2026-09-19, Wrong data shown or stored, `a61eeca` PR #42

- **What was seen.** Legitimate adult opt-ins to the data sharing programme were silently revoked.
- **Why.** Postgres fires same-timing triggers in alphabetical order by name. `profiles_sharing_eligibility` sorted before `profiles_sync_age_tier`, so eligibility was judged against a stale age tier.
- **How it surfaced.** Testing the guarantee the triggers were supposed to provide, rather than reading the trigger code. (A test caught it)
- **Fix.** Numeric prefixes on trigger names to force the order.
- **What stops it now.** `src/sql` smoke tests assert the guarantee, not the implementation in `supabase/migrations/20260918203431_fix_profiles_trigger_firing_order.sql`
- **Cost.** every adult opt-in, silently
- **Lesson.** Postgres fires same-timing triggers alphabetically. If two triggers on one table have an order dependency, encode it in the name, and test the outcome rather than the code.

INC-0013. UPDATE OF fires on the columns named in the statement, not the ones that changed

2026-09-19, Wrong data shown or stored, `a61eeca` PR #42

- **What was seen.** Updating `birth_year` stored a minor's tier and skipped the eligibility check entirely.

- **Why.** UPDATE OF <columns> keys off the columns named in the UPDATE statement, not off what actually changed and not off what an earlier BEFORE trigger wrote into NEW.
- **How it surfaced.** Same guarantee test as INC-0012. (A test caught it)
- **Fix.** Fire on every update and compare OLD to NEW inside the function.
- **What stops it now.** sharing eligibility fires on every update in `supabase/migrations/20260918204223_sharing_eligibility_fires_on_every_update.sql`
- **Cost.** a minor could pass the adult gate
- **Lesson.** UPDATE OF is a statement-shape filter, not a change filter. If you need 'when this value changed', compare OLD and NEW yourself.

INC-0020. Revoking EXECUTE from anon did nothing while PUBLIC still held it

2026-09-19, Wrong data shown or stored, `a61eeca` PR #42

- **What was seen.** Functions stayed callable after their grants were revoked from anon and authenticated.
- **Why.** PUBLIC is a separate grantee. A leading =X/postgres in pg_proc.proacl means PUBLIC holds EXECUTE, and revoking from the named roles changes nothing.
- **How it surfaced.** Re-running the Supabase advisors instead of trusting the per-role grants. (A test caught it)
- **Fix.** Revoke from PUBLIC as well, everywhere.
- **What stops it now.** every migration revokes from public, anon, authenticated together in `supabase/migrations/20260916_revoke_public_execute_and_rls_initplan.sql`
- **Cost.** an open function surface believed closed
- **Lesson.** In Postgres, revoking from every role you can name still leaves PUBLIC. Verify with the advisors or by reading the acl, never by reading your own migration.

INC-0056. Twelve profile fields said Saved and never reached the account

2026-09-21, Silent loss, PR #63

- **What was seen.** A signed-in user fills in education, intended major, score goal, application year, budget, industry, household income, first generation, military status, study hours or their birth date, sees a Saved toast, and none of it is on their account. It survives only in that browser.
- **Why.** Two layers disagreed and nothing compared them. Row Level Security scopes every write on the profiles table to the caller's own row, and a separate column grant decides which columns any role may write at all. The column grant to authenticated covers 5 of the 17 self-reported fields the Account page offers. The page updates the other 12 anyway, the database refuses, and aboutSave wraps the call in a try/catch with an empty body, so the refusal is discarded and the toast fires regardless.

- **How it surfaced.** Reading the column privileges while adding new profile fields for something else. Nothing in the product would ever have reported it. (Found by reading the code or the output)
- **Fix.** Grant UPDATE on the self-reported columns to authenticated, which is safe because the row policy already restricts every write to the caller's own row, and stop the save path swallowing its own error.
- **What stops it now.** a SQL smoke test asserts authenticated holds UPDATE on every field the Account page writes, and none of the server-owned ones in `src/sql/smoke_profile_grants.sql`
- **Cost.** every signed-in user's demographics, for as long as the fields have existed
- **Lesson.** An empty catch block around a write is a silent-loss defect waiting to be born. If a save can fail, the person must be told; a success toast that fires regardless of the result is worse than no toast, because it actively teaches the user the data is safe. And where two layers of authorisation have to agree, something has to compare them: the one that is wrong will not announce itself.

Search and metadata (3)

INC-0002. School URLs vanished from the sitemap when the data file was split

2026-08-24, Silent loss, `df14c7c` PR #23

- **What was seen.** Per-school pages were live and correct, and absent from sitemap.xml.
- **Why.** The sitemap generator read schools.json. When the library was split into one file per school, nothing read the new directory, and a loop over an empty list emits nothing rather than failing.
- **How it surfaced.** Noticed while reviewing the built sitemap. (Found by reading the code or the output)
- **Fix.** Generate sitemap entries from the same directory scan the pages are built from.
- **What stops it now.** the build asserts the sitemap URL count matches the page count in `src/build.py`
- **Cost.** unknown period of missing indexation
- **Lesson.** A refactor that moves data has to be followed to every reader, and a loop over nothing is the quietest failure in programming. Derive counts from one source and assert they agree.

INC-0014. A hardcoded count in the meta description went stale, and Google showed it

2026-09-19, Wrong data shown or stored, `36b1e92` PR #43

- **What was seen.** The meta description claimed 1,451 colleges after the number changed. `og:description` and `twitter:description` both derive from that tag, so all three were wrong.
- **Why.** A figure typed into a template instead of computed at build time, which is the exact failure build-time counting exists to prevent.

- **How it surfaced.** Reading the built page. (Found by reading the code or the output)
- **Fix.** Derive the number from the same count the pages are built from.
- **What stops it now.** no hardcoded corpus counts in templates in `src/build_rankings.py`
- **Cost.** a wrong figure in the search result snippet
- **Lesson.** Any number in user-facing copy that describes the size of something must be computed from that thing at build time. The moment it is typed, it has a half-life.

INC-0049. Two pages told the same story with different numbers after a reweighting

2026-09-19, Wrong data shown or stored, `a34a798` PR #47

- **What was seen.** The rankings index still showed figures from before a scoring change while the methodology page showed the new ones.
- **Why.** Two artefacts derived from one model, and only one was regenerated.
- **How it surfaced.** Reading both pages after the change. (Found by reading the code or the output)
- **Fix.** Derive both from the same computed values at build time.
- **What stops it now.** both figures are printed by the build from one source in `src/build_rankings.py`
- **Cost.** a visible contradiction between two pages
- **Lesson.** When one model feeds two pages, generate both from the model in the same pass. Two places that must agree will not, and the reader who notices is the reader you were trying to convince.

Interface and data display (3)

INC-0034. A line chart interpolated between discrete money events

2026-09-21, Degraded, `8827e64` PR #62

- **What was seen.** A line drawn between \$13.32 on the 18th and \$4.99 on the 19th, passing through every value in between.
- **Why.** Money movement was charted as a time series. A line encodes continuity, and these are discrete events on particular days.
- **How it surfaced.** Rendering it and looking at it. (Found by rendering it and looking)
- **Fix.** Bars for movement and cash. Lines are reserved for measures that genuinely vary continuously.
- **What stops it now.** Nothing automated. This one is still carried by attention.
- **Cost.** a chart that implied numbers that never happened
- **Lesson.** Chart form is a claim about the data. A line claims the values in between existed. Ask whether that claim is true before choosing it.

INC-0052. Two games signalled right and wrong by colour alone

2026-09-21, Degraded, `b4742a7` PR #57

- **What was seen.** Outcomes in two timed games were distinguishable only by colour, and nothing was announced.
- **Why.** The games were built for speed and the feedback was styling rather than content.
- **How it surfaced.** An accessibility pass over the games. (Found by reading the code or the output)
- **Fix.** Announce outcomes in a live region, and give a flipped tile a spoken identity. Colour is now secondary to a word in both.
- **What stops it now.** smoke tests assert the live region announces outcomes in `src/app_template.html`
- **Cost.** two games unusable with a screen reader
- **Lesson.** Never encode a state by colour alone. The word also survives greyscale printing, forced-colors mode and a glance from across a room, so it is better for everyone and not only for the people it is required by.

INC-0058. The fit card told readers to fill in fields that were below it

2026-09-22, Cosmetic, PR #64

- **What was seen.** With nothing entered, the comparison card said to fill the three inputs in above, and they are further down the same page.
- **Why.** The copy was written while building the card, before deciding where the inputs would live, and was never re-read against the rendered page.
- **How it surfaced.** Screenshotting the page and reading it as a user would. (Found by rendering it and looking)
- **Fix.** Name the section rather than a direction, and assert the wording in the test so a later move of either block fails loudly.
- **What stops it now.** smoke_fit asserts the card points at the section by name in `src/smoke_fit.js`
- **Cost.** a sentence sending users the wrong way
- **Lesson.** Copy that says above, below, left or right is a hard dependency on layout that nothing checks. Name the thing instead, and the sentence survives every rearrangement.

The Checklist

Generated from the defect ledger. Every line exists because something went wrong once. Nothing is here for completeness.

Read it before starting a piece of work in the matching area, and again before you push.

Build system

- [] A regex that counts things assumes a formatting convention, and a file that legitimately breaks the convention counts as zero rather than as an error. Any counter that can return zero for a non-empty input needs a per-source assertion, not just a total.

<small>The item counter missed a whole bank file because it assumed a quoting style (INC-0059)</small>

- [] A parse guard covers the file shapes someone thought of. When the same code moves into a new shape, a separate file, a chunk, a worker, the guard does not follow it. List what the guard covers against what the deploy actually ships, and check the difference rather than the intention.

<small>Nothing parsed the one file every user downloads (INC-0060)</small>

- [] A guard keyed to wording is a guard on the wording, not the fact, and every synonym is a hole in it. Widening the wording is the obvious repair and it trades missed defects for false alarms, which cost more because they get the guard switched off. Match a phrase that only the thing you care about can produce, rather than every word it might happen to use.

<small>The stale count guard checked two nouns and the page used a third (INC-0063)</small>

- [] A guard that covers a subset of cases reproduces the original defect in the cases it skips, and it is more dangerous than no guard because the incident it was written for feels closed. When you add a check, enumerate everything of that kind and cover all of it, or state in the code which cases are deliberately excluded and why.

<small>The guard against a blind counter was itself blind to three exams (INC-0064)</small>

- [] Two habits, both mine rather than the code's. Verify with the sequence the pipeline runs, read out of its config, not with the subset you remember: a suite chosen from memory drifts to the parts that were failing last week. And when a step is deliberately non fatal, the word it fails with is the whole of its signal, so it has to be the word people grep for. WARNING on a line that means a deliverable did not build is an invitation to miss it, and the cost of saying ERROR while still exiting zero is nothing at all.

<small>A build step that fails while the build exits zero, and a verification run that was a remembered subset (INC-0080)</small>

- [] A size limit on a generated file is only a guard if something bounds the generator too; otherwise it is a delayed failure that lands on whoever commits next, and reads as their fault. When two guards constrain the same output, check the fix against both: shortening this file to satisfy the size check would have broken the completeness check that reads its first 60 characters.

<small>The rules digest promises to be prompt sized and its generator grows without bound (INC-0083)</small>

CSS and layout

- [] The same undefined-property failure will find you repeatedly, at every severity from one icon to an invisible legal control. One audit of computed colour catches the whole class.

<small>A landing-page icon referenced a colour token that did not exist (INC-0050)</small>

- [] CSS comments do not nest. When a whole page category loses its styling, read the built artefact, not the source that produced it.

<small>CSS comments do not nest, and one placeholder killed the palette on 1451 pages (INC-0010)</small>

- [] CSS never fails loudly. A malformed rule is skipped, a bad selector eats the block after it, and an undefined variable paints as nothing. Anything that matters visually has to be asserted on the rendered page, because the parser will not tell you.

<small>Error recovery in the CSS parser swallowed the palette silently (INC-0048)</small>

- [] An undefined CSS custom property is silent. Audit computed colour, not authored colour, and do it on the rendered page.

<small>The Do Not Sell button was invisible from the day it shipped (INC-0018)</small>

- [] A dark mode does not break by having a wrong colour. It breaks by missing one. Any colour paired with a ramp has to move with the ramp.

<small>A sequential colour ramp inverts direction between themes, and the label text did not (INC-0029)</small>

Content generation

- [] Any generator that claims reproducibility must be seeded from something stable across processes. `hash()` is not, in Python, and the failure shows up as a flaky test rather than as a wrong answer.

<small>Item banks were different on every build because Python randomises `hash()` (INC-0003)</small>

- [] Deletion by shadowing is invisible. Any collection whose size is a fact about the product needs its size asserted, not just its contents.

<small>A shadowed variable silently deleted 3000 items (INC-0004)</small>

- [] In any set of multiple-choice content, count where the answers are. A positional tell makes the whole set worthless to a test-wise user, and it is invisible item by item.

<small>225 of 302 correct answers sat at position A (INC-0039)</small>

- [] Test your content against the strategies a lazy adversary would use, not only against whether it is correct. Measure the score of a rule that ignores the question.

<small>The longest option was the correct answer 81 percent of the time (INC-0044)</small>

- [] A deduplication key must be canonical under every transformation the item legitimately undergoes. Ask what varies per draw before you hash.

<small>The dedup key counted a reshuffled question as a new one (INC-0007)</small>

- [] The same inflation arrives through a different door every time you close one. When you fix a dedup bug, ask what else shares an identity.

<small>Four passages produced five hundred fake distinct questions (INC-0008)</small>

- [] Dedup can be wrong in both directions. An over-broad key deletes real content as silently as a narrow one inflates it.

<small>Two conditionals hashed identically and half the inference items would have vanished (INC-0009)</small>

- [] Every filter needs its rejection count reported. A filter that silently drops is indistinguishable from an input that was never there.

<small>A length guard silently dropped sixteen valid items (INC-0011)</small>

- [] A guard on the extreme of a distribution can be satisfied by moving the mass next to the extreme. When you correct for a measured bias, measure the whole distribution afterwards, not the statistic you were correcting.

<small>Correcting a length tell moves it one rank over, every time (INC-0062)</small>

- [] A correction table is a set of claims about outcomes, and an entry that quietly fails still counts as applied. Aggregate metrics hide this well: a table where half the entries work still moves the number in the right direction, which reads as success. State the per item intent in a form the machine can check, and every entry that did nothing says so by name.

<small>Half the length tell correction did nothing and the build said it had worked (INC-0066)</small>

- [] A seeded shuffle is deterministic, which makes calling it twice look harmless: the same input gives the same output. What repeats is the permutation, not the randomisation, and a permutation applied to its own result is biased toward leaving things where they were. Any function whose value comes from being applied exactly once should refuse to be applied twice rather than relying on the caller to remember. The second lesson is about reading: the generator printed the defect on every run, above the line being watched.

<small>Shuffling the answers twice put 37 percent of the keys at A (INC-0068)</small>

- [] An aggregate over a mixed population reports the population, and if part of that population is flat by construction it will hide the part that is not. The rule that follows is about what the unit of the measurement should be: measure at the grain the defect can exist at, which here is the file, because a file is written by one person in one sitting with one set of habits. The section was the grain the data was convenient at.

<small>A bank a student can play at 88 percent, inside a section the check passed (INC-0069)</small>

- [] Writing a second tool for the same job in a different context reproduces every detail the first one learned the hard way, unless the detail is written down somewhere the second author will look. This is INC-0067 seen from the other side: there the callers were not migrated to the helper, here the helper's behaviour was not carried into the second implementation. Both are the cost of a rule living in code rather than in a statement of the rule.

<small>The second tool that appends a clause did not carry the first one's rule about the full stop (INC-0070)</small>

- [] A report that truncates its output invites the reader to write text that continues it, and a tool that appends will put that text somewhere else. Either the report should not truncate the field the caller has to write against, or the tool should refuse input shaped like a continuation. The cheap half is the refusal, because it is one condition and it cannot be forgotten, while remembering not to write continuations is a habit that has to hold every time.

<small>Clauses written from a truncated report were appended to the end of the wrong word (INC-0071)</small>

- [] A record has parts that refer to one another, and a tool that edits one part by text is editing a graph while looking at a string. The cheap guard is not to check every reference but to refuse the edit when the old text occurs anywhere else in the record, because that is the only place a reference to it can be. Refusing on a false positive costs one rewritten table entry; not refusing ships an explanation about an option nobody saw.

<small>A distractor was replaced and the explanation went on naming the old one (INC-0072)</small>

- [] A guard that takes the intent as an argument is only as good as the argument, and an argument derived by hand from the same data the guard is checking is a second implementation of the thing being checked. It fails in the direction that is hardest to see: too high an intent demands a rank the clauses cannot reach, and the author satisfies it by writing more clauses than the plan called for, which skews the distribution the other way while every check passes. Derive the intent from the data with the code that already reads it.

<small>check_lift was told a one clause entry lifted two distractors (INC-0073)</small>

- [] A corpus field is written against the one sentence the author had in mind, and the schema that reuses it three templates later has no way to know which shape it is. The type system says str in both places. Two things follow. Store the field in every shape a template needs and name the shapes, rather than storing one shape and trusting the next author to notice. And guard the output, not the corpus: the generated sentence is the only place the mismatch becomes visible, and a cheap pattern over the rendered text catches a class that no check on the inputs can see.

<small>A corpus field written for one grammatical slot was spliced into another (INC-0074)</small>

- [] A guard written from the instance in front of you covers that instance. INC-0074 was a bare infinitive in a noun slot, so the guard looked for bare infinitives, and the sentence one screen away in the same file was a wh clause in a clause slot and went straight through. The general defect was never the infinitive; it was that a corpus field carries no record of the grammatical shape it was written in, and any template may reuse it. So the guard has to be stated over the class, every field against every slot, not over the token that happened to be wrong first. The other half of this is where it was found: the distractor version was spotted first because it is louder, and the version in the key, which is three times as damaging, was found only because the first one prompted a second look. Reading one rendered item per schema would have caught both on the day they were written, and costs less than either fix.

<small>Every correct answer on one GMAT schema was ungrammatical, and the guard written an hour earlier could not see it (INC-0075)</small>

- [] A standard library function whose name is a plausible description of half of what it does will be used for that half. `capitalize()` reads as "make this the start of a sentence" and is in fact "make this the start of a sentence and flatten everything else", and the damage is invisible until a value happens to contain a capital. The guard is not a test that the output looks right, because the output looked right for every value that had no name in it. The guard is to ban the function: the correct one is three characters of slicing, the wrong one is never what a generator wants, and a lint catches it in the diff rather than in the bank.

<small>str.capitalize() lowercased the rest of the sentence, and took the proper nouns with it (INC-0076)</small>

- [] Presentation rules travel with the value, and a value formatted at the point of use is formatted by whoever was writing that line. Eight schemas each wrote the same two characters and all eight omitted the same exception, which is not eight mistakes but one missing function. The give away is the shape of the defect: identical output in unrelated files means the knowledge was never in one place. The check that catches it cannot be on the arithmetic, because the arithmetic was always right, so it has to be on the rendered string.

<small>A coefficient of one was printed as 1x on 2,353 shipped items (INC-0078)</small>

- [] A check is scoped to a grain, and the grain is a claim about where a defect can live. INC-0069 moved the grain from the section to the file for hand written banks and stopped there, so the same defect went on living one level down in generated ones, where there are far more items. When a check finds something by being made finer, the question to ask immediately is what else is measured at the old grain. The second half is about which populations a check can see: this one ran on what the test harness loads, which is a sample chosen for a different purpose, and a sample chosen for a different purpose is not a population you can make claims about.

<small>A generated schema was playable at 100 percent by picking the third shortest option, and no check looked at generated schemas (INC-0079)</small>

- [] Every guard here measured the answer's place in its set, and a set of guards that all take the same kind of measurement shares a blind spot the size of everything else. The tell they could not see was the simplest one a student would find: the answer is the same answer. When adding the third check of a kind, the question worth asking is not whether it is stricter than the other two but what all three have in common, because that is what is going unmeasured.

<small>A student who always answers 1 scores 98 percent on a schema, and no check looked at the answer itself (INC-0081)</small>

- [] Two lessons, and they compound. A rule copied into code by its examples loses the clause the examples were illustrating: CLAUDE.md bans six named sites and coaching site blogs, and the list kept the six and dropped the category, which is the half that generalises. And a validator gets written for the corpus that had the problem at the time, then quietly defines what is checked: two of three published corpora were enforced and the third had never had a source read, which is not a weaker check but an absent one. When a guard exists, the question is not only whether it is strict enough but which of the things it could be pointed at it is not pointed at.

<small>Nine published exam facts cite test prep companies, in the one published corpus with no source validator (INC-0082)</small>

Database

- [] Postgres fires same-timing triggers alphabetically. If two triggers on one table have an order dependency, encode it in the name, and test the outcome rather than the code.

<small>Postgres fired triggers alphabetically and revoked every adult opt-in (INC-0012)</small>

- [] UPDATE OF is a statement-shape filter, not a change filter. If you need 'when this value changed', compare OLD and NEW yourself.

<small>UPDATE OF fires on the columns named in the statement, not the ones that changed (INC-0013)</small>

- [] In Postgres, revoking from every role you can name still leaves PUBLIC. Verify with the advisors or by reading the acl, never by reading your own migration.

<small>Revoking EXECUTE from anon did nothing while PUBLIC still held it (INC-0020)</small>

- [] An empty catch block around a write is a silent-loss defect waiting to be born. If a save can fail, the person must be told; a success toast that fires regardless of the result is worse than no toast, because it actively teaches the user the data is safe. And where two layers of authorisation have to agree, something has to compare them: the one that is wrong will not announce itself.

<small>Twelve profile fields said Saved and never reached the account (INC-0056)</small>

Front end

- [] Generated code is code. If your build writes JavaScript into a string, the build must parse the result, because the blast radius of one bad character is the whole file, not the line.

<small>Unescaped quotes in onclick strings took the whole app down (INC-0001)</small>

- [] The moment a single-tenant store becomes multi-tenant, every key in it is a collision waiting to happen. Enumerate the writers before you add the second tenant, not after.

<small>One exam's progress blob overwrote another's (INC-0038)</small>

- [] A conditional that treats not-A as the original case is a bug the day a third case exists. Resolve variants from data, and the third one costs a row rather than a search.

<small>The GRE app told GRE students to calibrate at the wrong test maker's site (INC-0043)</small>

- [] Nobody notices a page getting slower one commit at a time. Put the number in a test the first time you care about it, not the first time somebody complains.

<small>The whole bank blocked first paint: 20 seconds to the first question on 3G (INC-0015)</small>

- [] try/catch around an API that returns errors is decoration. Know which convention each call uses before you wrap it.

<small>A try/catch caught nothing because the call inside it does not throw (INC-0022)</small>

- [] When you change a filename scheme, grep for the old name as a string, including inside regexes. A pattern is a hardcoded name wearing a disguise.

<small>The service worker precache regex did not match the chunked files (INC-0028)</small>

- [] A fixed rounding rule is wrong at one end of the range or the other. Pick the precision from the magnitude, and write the expected number down before you write the code that produces it.

<small>ARR was rounded to whole dollars and lost real money at small scale (INC-0032)</small>

- [] Anything that reports failures must not be able to report its own. Check whether each call rejects or throws before you wrap it, and make the reporting path unable to re-enter itself.

<small>The error reporter reported its own failures, in a loop (INC-0045)</small>

Infrastructure and deploy

- [] Two hostnames are two origins and therefore two of everything the browser scopes by origin. Pick one and redirect on the server, not in a meta tag.

<small>www and the apex were two origins, so consent and rankings split in half (INC-0023)</small>

- [] An exclusion list is a denylist, and denylists are wrong by omission. Derive the allowed set from what the built pages actually reference.

<small>The .git directory was being served on production (INC-0024)</small>

- [] Verify a cache-fronted fix with a cache-busting request, or you are testing the cache. This costs one query parameter and saves an hour of chasing a fix that already worked.

<small>A stale edge cache made a fixed 404 look like a live 200 (INC-0025)</small>

- [] Know your platform's hard limits and assert them in the build. A deploy-time rejection is a bad place to learn a number your build could have told you.

<small>A 29.88 MiB bank file would have failed the deploy the moment it merged (INC-0027)</small>

- [] A guard that is a hand-kept list of safe paths is written by the same hand that made the mistake. Derive the safe set from the artefact, not from memory.

<small>Excluding the design directory would have shipped the legal pages with no styling (INC-0047)</small>

Interface and data display

- [] Chart form is a claim about the data. A line claims the values in between existed. Ask whether that claim is true before choosing it.

<small>A line chart interpolated between discrete money events (INC-0034)</small>

- [] Never encode a state by colour alone. The word also survives greyscale printing, forced-colors mode and a glance from across a room, so it is better for everyone and not only for the people it is required by.

<small>Two games signalled right and wrong by colour alone (INC-0052)</small>

- [] Copy that says above, below, left or right is a hard dependency on layout that nothing checks. Name the thing instead, and the sentence survives every rearrangement.

<small>The fit card told readers to fill in fields that were below it (INC-0058)</small>

Payments

- [] Row Level Security is row-level. Which columns a role may write is a separate grant, and anything money depends on belongs to the service role alone.

<small>A signed-in user could grant themselves a paid plan (INC-0041)</small>

- [] Enumerate every value a third-party status field can take before you branch on one of them. The value you did not think of is usually the one that matters commercially.

<small>Every trialing subscriber would have been left on the free plan (INC-0042)</small>

- [] Idempotency belongs in the database, not in the handler. Handlers race; unique indexes do not.

<small>Two Stripe event types announce one new subscription (INC-0035)</small>

- [] Read what a payment field means, not what it is called. Cumulative and incremental fields look identical until the second event.

<small>charge.amount_refunded is cumulative, so partial refunds double-count (INC-0036)</small>

- [] When a write is destructive, capture what you need from the old value first. Ask what question you will want to answer after this row is gone.

<small>Cancellation erases the number you need to record the cancellation (INC-0037)</small>

Scoring and selection

- [] If your system branches on difficulty, measure that the branches actually differ. A label is not a property.

<small>The easier module was not easier (INC-0040)</small>

- [] Check that your instrumentation fired at all before you trust anything built on it. An empty table looks identical to a quiet week.

<small>The adaptive engine was learning from nobody (INC-0005)</small>

- [] A type system spread across a renderer and a grader will drift. The cheapest guard is one that exercises every variant end to end, once.

<small>Sentence equivalence items were unanswerable however well you answered (INC-0006)</small>

- [] When you add a filter, find every path that adds items after the filter runs. A gate on the entry point is not a gate on the set.

<small>Fixing repeats directly left a side door through passage groups (INC-0051)</small>

Search and metadata

- [] A refactor that moves data has to be followed to every reader, and a loop over nothing is the quietest failure in programming. Derive counts from one source and assert they agree.

<small>School URLs vanished from the sitemap when the data file was split (INC-0002)</small>

- [] Any number in user-facing copy that describes the size of something must be computed from that thing at build time. The moment it is typed, it has a half-life.

<small>A hardcoded count in the meta description went stale, and Google showed it (INC-0014)</small>

- [] When one model feeds two pages, generate both from the model in the same pass. Two places that must agree will not, and the reader who notices is the reader you were trying to convince.

<small>Two pages told the same story with different numbers after a reweighting (INC-0049)</small>

Tests and guards

- [] Measure the moment the user can act, not a browser lifecycle event. A test that measures the wrong instant is worse than no test, because it produces a number people trust.

<small>The performance test waited for the load event, which waits for the thing being optimised (INC-0016)</small>

- [] A regex with a length bound is a guard with an expiry date. Assert the number of things checked, not only that the checks passed.

<small>A build guard silently narrowed its own scope when the counts grew (INC-0017)</small>

- [] Check your checker. A tool that cries wolf gets muted, and then it is worse than nothing.

<small>The audit reported a working link as broken (INC-0019)</small>

- [] A negative assertion passes when the system is broken in the right way. Always pair it with the positive case, or it is testing nothing.

<small>A test passed because nothing was configured at all (INC-0021)</small>

- [] A stochastic measurement on one seed is an anecdote. If your system has randomness, a single-run before-and-after cannot distinguish a change from the weather.

<small>I reported a regression that was seed noise (INC-0026)</small>

- [] Classify by what a thing is, not by what it is called. Naming conventions are a hint, never a type.

<small>The theme parity test matched by name prefix and caught a font size (INC-0030)</small>

- [] A test that is not wired into CI is a test that does not exist. Prove a suite runs in the place it is supposed to run, not on your machine.

<small>The browser suites had never actually run, because npm install had not (INC-0031)</small>

- [] Before you measure a layout, assert the thing is rendered. Hidden elements answer most DOM questions, and they answer them wrongly.

<small>The dashboard test measured a hidden element and passed (INC-0033)</small>

- [] An error feed with no filter is a feed nobody reads. Signal has to be defended, and the cheapest defence is a human label that mutes permanently, so the queue gets quieter as it learns.

<small>The error queue was 81 percent other people's blocked scripts (INC-0046)</small>

- [] A guard that reads the repository needs the repository. CI checkouts are shallow by default, and anything that walks history, blames a line or resolves an old hash will fail in a way that looks like the data is wrong rather than the clone.

<small>The playbook's own citation guard failed CI on its first run (INC-0053)</small>

- [] When you add a condition that skips a check, make sure it describes the failure and not something merely correlated with it. A skip is indistinguishable from a pass in the output, so the fix for a noisy check can silently delete it.

<small>The first fix asked the wrong question and muted a working check (INC-0054)</small>

- [] A path that exists on the machine you wrote the test on is not a path. Resolve environment-specific locations through one helper that falls back to the tool's own default, and return undefined rather than an empty string, because undefined means 'you decide' and an empty string means 'launch nothing'.

<small>A new browser suite hardcoded this machine's browser directory and crashed in CI (INC-0055)</small>

- [] A commit hash is not a durable citation in a repository that squashes. Pull requests, issues and tags survive history rewriting; branch commits do not. Cite the thing that outlives the merge, and make any check of the other one advisory.

<small>The ledger cited commits that squash merging destroys (INC-0057)</small>

- [] A metric that moves against you when the product improves will eventually be used to justify reverting an improvement. When a number goes the wrong way after a change that should only have helped, measure the underlying thing directly before believing either the number or your own explanation of it. Never redefine the metric in the same change that made it look bad.

<small>A repeat metric that gets worse when the bank gets better (INC-0061)</small>

- [] An aggregate is a claim about whatever you grouped by. Group by the file and you have measured the file. State the grouping in the sentence that reports the result, and the overclaim becomes visible while you are writing it.

<small>The analysis chapter called one file eight failing guards (INC-0065)</small>

- [] Extracting a shared helper does not migrate the callers. The extraction fixes the file it was extracted from and leaves every sibling on the old path, which is INC-0059 and INC-0064 in a different costume: a correction applied to the instances in hand rather than to the pattern. Three suites had failed visibly and ten were wrong; the seven silent ones were found by the guard, not by reading. When a helper exists because a direct call was wrong, make the direct call fail the build, and let it enumerate the callers rather than enumerating them by hand.

<small>The browser path fix covered two suites and three others kept crashing (INC-0067)</small>

- [] A check that reports pass or fail from a handful of random draws is a check that will flip on work that has nothing to do with it, and the cost is not the false alarm. It is that the next real alarm arrives in a tool people have learned to shrug at. Before believing or dismissing a warning, run the thing it measures enough times to know its rate: that answers both whether this alarm is real and whether the check is worth keeping in its current form. Here the answer was that the engine was fine and the check was wrong, and both were worth knowing.

<small>A review bot check whose verdict was three coin flips warned on an unrelated bank change (INC-0077)</small>

Adapting This to a Different Business

Most of this book is about a test-preparation platform. This chapter separates what was specific to that from what was structural, so you know what to keep.

What was specific

The item response theory, the exam registries, the score bands, the school library, the concordance between two editions of one exam. If you are not building an assessment product, none of it applies directly.

What was structural, and transfers unchanged

The infrastructure. Edge-hosted static pages, managed Postgres with Row Level Security, serverless functions, hosted checkout. Under thirty dollars a month, and it serves a hundred thousand generated items and fifteen hundred generated pages.

The build as the guard rail. Sources in, pages out, and every guard living in the build because the build is the only thing that sees the artefact.

Authorisation in the database. RLS everywhere, privileged reads behind definer functions, revoke from `PUBLIC`, never take an id from a request body.

The event-ledger pattern. Any external system that sends you events sends them out of order, more than once, and sometimes twice under different names. Never write the payload; re-read state. Put idempotency in a unique index. Keep an append-only record of facts alongside whatever current-state row you maintain, because current state is destroyed on every update.

The honesty discipline. Every published figure carries a source, a year and a URL. Unverifiable is null, never a guess. Never present an internal threshold as official. Never show a number about a person without its error bar. Decline to compute a metric you cannot compute honestly, and say why on the page.

The interface system. One token file, three typefaces, negative tracking on headings, `min-width: 0` on every grid and flex child, a complete palette on bare `:root` with only overrides in the theme blocks, colours in charts validated under simulated colour vision deficiency rather than chosen.

The test ladder and its failure modes. Every one of the five ways a test can be silently wrong applies to any codebase.

The defect ledger itself. The single most transferable artefact in this repository is `data/playbook/incidents.jsonl` and the script that turns it into this book.

The substitution table

THIS PROJECT	YOURS	KEEP
Item bank generators	Whatever your content is	Dedup keys canonical under every legitimate transformation; counts reported per stage; filters report rejections
Exam registry	Any multi-tenant or multi-variant dimension	Resolve differences from data, never from <code>if (x === 'a')</code> , which treats "not a" as "the original"
Ability estimation	Any inferred number about a user	A range not a point, a floor on the interval, a minimum evidence threshold, the method published
School library	Any library of external facts	One file per entity, source and year and URL on every figure, banned-source list enforced at build time
Two exam editions	Any legacy unit beside a current one	Never convert, compare on a common normalised measure, cite the concordance
Stripe plus Apple	Any two sources of one truth	Leave the first alone, add a derivation on top, move readers to the derivation
Forum moderation	Any user-generated content	Automatic screen flags, a human decides, the decision is stored and mutes permanently

The first week, for something new

1. Write the rules file. Use the template in the bootstrap pack and fill in the four

blanks.

1. Create the repository, the build script, and one page. Deploy that page to the real

domain.

1. Learn the platform's limits now: per-file size, which directories the uploader walks,

whether `www` and the apex are one origin, whether your headers file applies to responses your own code generates.

1. Stand up the database with RLS on every table from creation, and write one SQL smoke test that asserts a guarantee and rolls itself back.

1. Write the token file. All of it, both themes, before the second page exists.
2. Then build the product.

The one paragraph version

Make failure loud before you make anything else. Put every guarantee in the layer that cannot be bypassed: the database for authorisation, the build for correctness of the artefact, a pinned number for anything statistical. Never write a figure you cannot regenerate. Never show a number you cannot defend. When something breaks, write down what you believed five minutes earlier, because that belief is the actual defect and it is the first thing you will lose.

The Bootstrap Pack: Starting a New Project With This Loaded

This chapter answers one question: what do you actually paste into a fresh Claude conversation so that a new build starts with everything this one learned?

The wrong answer, first

Do not paste the book. It is roughly the length of a short novel, and pasting it has three problems. It consumes context that the actual work needs. It buries the twenty operative rules inside several thousand sentences of reasoning about a test-prep site. And most of it is *explanation*, which is what you want when you are deciding something and noise when you are executing.

Reference material and operating instructions are different artefacts and need different delivery.

The book is reference. The rules are instructions. Only the second goes in the prompt.

The pack

`python3 src/build_playbook.py` writes four files into `playbook/bootstrap/`, all generated from the same ledger as the book, so none of them can drift from it.

CLAUDE.template.md goes in the new repository's root. Claude Code reads it on every turn. It is the constitution: the house rules, the never-invent-a-value rule, the sourcing rules, the communication rule, and a short architecture section you fill in as you build. Four bracketed blanks at the top are the only things you have to change.

KICKOFF.md is the literal first message for a new conversation. It sets the working agreement, the phase order, and what a session should report. Paste it, then describe your business idea underneath it.

RULES_DIGEST.md is every lesson in the defect ledger, compressed to one line each and grouped by area. It is about three pages. This is the highest value-per-token artefact in the whole project: 83 real defects reduced to the rules that prevent them, with the specifics of this codebase stripped out.

incidents.jsonl is the raw ledger, copied so the new project can start appending to it on day one rather than starting an empty one.

How to load it, by surface

Claude Code, new repository. Copy `CLAUDE.template.md` to the repo root as `CLAUDE.md`, fill in the four blanks, and copy `incidents.jsonl` to `data/playbook/incidents.jsonl`. Copy `src/build_playbook.py` too, so the new project's own ledger becomes its own book from the first defect. Then open a session with `KICKOFF.md` as the first message.

A Claude Project on claude.ai. Put `BUILD_PLAYBOOK.md` and `RULES_DIGEST.md` into Project Knowledge, where they are retrieved on demand rather than sitting in every message. Put the contents of `CLAUDE.template.md`, filled in, into the project's custom instructions. Start the first conversation with `KICKOFF.md`.

A single conversation, no project. Paste `RULES_DIGEST.md`, then `KICKOFF.md`, then your idea. Attach `BUILD_PLAYBOOK.md` as a file if the surface allows it. That ordering matters: rules first, so they frame everything after, and the long reference last or not at all.

Why this layering works

Three reasons, and they are worth understanding rather than just following.

Retrieval beats repetition. A document in project knowledge is consulted when relevant. The same document in the prompt is re-read on every single turn, at full cost, mostly when it is irrelevant. For anything longer than a few pages, retrieval is strictly better.

Instructions need to be short enough to be followed. A rule competing with ten thousand words of context is a rule that gets applied inconsistently. The digest is deliberately brutal: one line per lesson, imperative mood, no examples. The examples are in the book, where they can be looked up when a rule seems wrong.

The ledger is the part that compounds. The recipe chapters age. The rules do not, because each one is the residue of a real failure, and the failure modes of software are considerably more stable than its tooling. A new project that starts with 83 defects already prevented is genuinely ahead, and every defect it hits of its own makes the next project further ahead still.

Keeping the loop closed

The pack is only worth carrying forward if the new project feeds it back. So the working agreement in `KICKOFF.md` includes one instruction that does the whole job:

> When something breaks, append a record to `data/playbook/incidents.jsonl` before fixing > it, while you still remember what you believed was true five minutes ago.

That is the entire maintenance burden. Everything else, the checklist, the analysis, the digest, the book, is generated from those records.

What to change for a different domain

`CLAUDE.template.md` has four blanks:

1. **The product, in one sentence**, and who it is for.
2. **The house style rules you will enforce**. Keep the dash ban unless you have a reason

not to; it is free and it makes the build's style enforcement real.

1. **The sourcing regime**. If you publish external facts, keep the whole thing including the banned-source list. If you do not, delete that block rather than leaving it as decoration.

1. **The architecture section**. Empty at the start. Fill it in as decisions are made, and only with things that are expensive to rediscover.

Everything else in the template is domain-independent and should be kept verbatim.